MESS: Fast and Private Semantic Search on Multi-Graph HNSW

Haoyu Cui
cuihaoyu@mail.sdu.edu.cn
Shandong University
China

Zengpeng Li
lizengpeng@sdu.edu.cn
Shandong University
China

Tien Tuan Anh Dinh
anh.dinh@deakin.edu.au
Deakin University
Australia

Mei Wang
wangmeiz@sdu.edu.cn
Shandong University
China

Abstract

Semantic search systems map data to a high-dimensional vector space and support retrieval of similar data via approximate nearest neighbor search. When the system is hosted by a trusted cloud provider, there is no privacy for the data or the query. Our goal is to design a system with three properties: privacy, accuracy, and efficiency. Existing works adopt either homomorphic encryption (HE), oblivious RAM (ORAM), or a differential privacy (DP) approach. They fall short of achieving all three properties.

In this paper, we present MESS, a system that realizes our goal. It maps the original vectors into binary codes, applies locality-sensitive hashing (LSH) and randomized response, and constructs a multi-graph Hierarchical Navigable Small World (HNSW) index over the perturbed codes. MESS ensures privacy of data, queries, and access patterns. It also ensures search pattern privacy via a two-phase query perturbation mechanism. The multi-graph index mitigates the impact of perturbation on result quality, thereby achieving accuracy. MESS is efficient because search is performed directly over perturbed codes, without the overhead of homomorphic encryption or ORAM. We provide a formal analysis of the system's privacy and an extensive evaluation of its performance. The results show that MESS achieves up to 15.08× lower latency than state-of-the-art baselines.

PVLDB Reference Format:
Haoyu Cui, Zengpeng Li, Tien Tuan Anh Dinh, and Mei Wang. MESS: Fast and Private Semantic Search on Multi-Graph HNSW. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code and supporting artifacts will be made publicly available upon acceptance.

1 Introduction

Semantic search systems support a wide range of applications, from classic information retrieval, image matching [33, 41], and recommendation [38] to pattern recognition [13, 47] and recent retrieval-augmented generation (RAG) applications. A typical semantic search system maps data and queries to high-dimensional vectors (or embeddings). Its main operation is approximate nearest neighbor (ANN) search, which finds the closest vectors to a given search vector using a distance metric. Recent RAG applications, driven by the success of large language models (LLMs), have led to renewed interest in ANN algorithms, indices, and vector database systems that enable efficient and scalable semantic search [20, 28, 46]. As data and query volumes grow, semantic search systems are migrating to the cloud [1–3]. Specifically, cloud-based semantic search services are attractive because they can be efficient, accurate, cost-effective, and scalable. However, they offer no privacy guarantees, as they operate on raw (or plaintext) embeddings and user queries, which can reveal sensitive information about the data and the user. As a consequence, they do not support applications that process sensitive data, such as private search over personal photos and documents and personal AI agents [25].

Our goal is to design a semantic search system with three properties: privacy, accuracy, and efficiency. The first property covers privacy of data, queries, access patterns, and search patterns. The second property means that the search results have high recall, while the last property means low search overhead. The main challenge in achieving this goal is to satisfy all three properties simultaneously. Existing works on private semantic search adopt one of the following three approaches. First, the homomorphic encryption and secure-computation approach, for example, [11, 29, 31, 37], computes vector distance directly on encrypted vectors. This approach achieves privacy but is inefficient due to the overhead of encrypted computation with high-dimensional vectors. Even when using clustering to reduce the search space, the search cost is still significant because of the large number of candidate vectors. Second, the ORAM approach, for example [4, 51], achieves privacy by hiding the vectors being accessed during search. In particular, Compass [51] combines an efficient vector index, ORAM, and product quantization to protect graph-access patterns [51]. However, it still fails to achieve efficiency because adaptive graph traversal requires multiple ORAM operations, increasing communication overhead. Third, the differential privacy (DP) approach, for example [8, 10, 16], trades privacy for efficiency by reducing strict privacy to quantified leakage and allowing distance computation on perturbed vectors. However, existing works do not consider any index, thus suffering from high search overhead. They fail to achieve accuracy due to recall degradation caused by vector perturbations.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

We present MESS that achieves our goal. It addresses the above challenge via a novel combination of differential privacy and a multi-graph index. In particular, it adopts the hierarchical navigable small world (HNSW) graph index for ANN search. It applies locality-sensitive hashing (LSH), followed by an LSH randomized response mechanism, to the original embeddings before building the graph. The DP mechanism ensures privacy of data and access patterns while enabling efficient computations over perturbed vectors. MESS overcomes the accuracy challenge of the DP-based approach by building and searching over multiple graphs, which reduces rank distortion and candidate set expansion. MESS introduces a two-phase query-perturbation process that restricts leakage of search patterns. Tab. 1 compares MESS with other works providing private keyword and semantic search. In summary, we make the following contributions.

- We propose an efficient HNSW search over perturbed binary embeddings. The client maps data and query vectors to binary codes, then applies an LSH randomized response mechanism to them. The server builds multiple HNSW graphs over the perturbed codes. It performs a Hamming-distance search on the graphs and returns the results in a single round.
- We propose a multi-graph design that addresses search quality degradation caused by perturbations, and a two-stage randomization process to protect repeated queries against averaging and direct linkage attacks. In particular, each query is routed to multiple graph indices and their results are aggregated. Furthermore, the client applies a permanent and an instantaneous randomization to each query.
- We provide formal analysis of the privacy bounds for the stored index, query access patterns, and repeated-query search patterns. We evaluate representative cross-shard inference methods to measure practical leakage.
- We evaluate MESS on standard datasets and compare it against non-private, homomorphic-encryption, and ORAM baselines. The results show that MESS achieves up to 15.08× lower latency and 35.28× lower communication overhead than Compass [51]. On the SIFT100M dataset, MESS achieves 52.53 ms/query, demonstrating its scalability.

The remainder of the paper is structured as follows. Sec. 2 presents the system model and discusses the security goals. Sec. 3 provides the relevant background, followed by Sec. 4, which describes MESS in detail. Sec. 5 presents the security analysis, followed by the performance evaluation in Sec. 6. Sec. 7 discusses related work, and Sec. 8 concludes.

2 System and Threat Models

In this section, we first outline the architecture of MESS. We then discuss the threat model, followed by the system and security goals.

2.1 System Overview

There are two entities in MESS: a client C and a server $\mathcal{S}$. The client is the data owner and user; that is, it outsources the data to S and issues search queries. The client's data is modeled as $\mathcal{D} = \{(\mathrm{id}_i, \boldsymbol{x}_i)\}_{i=1}^{N}$, where id_i is the data item i unique identifier, and $\boldsymbol{x}_i \in \mathcal{X}$ is its corresponding embedding vector. The client encrypts its data before sharing it with the server.

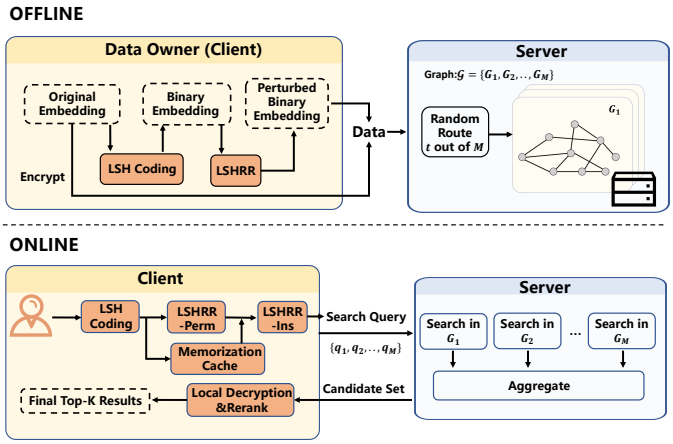


Fig. 1: System architecture of MESS.

Fig. 1 shows the system architecture of MESS. It operates in two phases. In the offline phase, the client maps the original embeddings to binary codes using locality-sensitive hashing (LSH) and perturbs them using LSH randomized response (LSHRR). The server maintains M HNSW graph indexes (or *shards*). The client encrypts each data tuple and uploads it together with the perturbed codes to a subset of M. The server builds the HNSW indexes based on the perturbed codes. In the online phase, the client hashes and perturbs the query embedding. The server performs a Hamming-distance search for nearest neighbors of the perturbed query embedding, across all the shards. It then aggregates the shard-level candidates and returns their encrypted data. The client decrypts the results, removes duplicates using the identifiers, and reranks the candidates based on the decrypted embeddings.

Scope. MESS supports semantic search over a database of vector embeddings, as opposed to directly over raw data (e.g., documents or images). We abstract away the data pre-processing pipeline that generates embeddings from raw data, and the post-processing pipeline for retrieving the raw data based on the search results. The embedding generation can be performed by off-the-shelf embedding models for different data modality ML models [23]. MESS can be deployed in a setting where the client keeps the original data locally and only uses the server to find similar data [51]. In this setting, the client uses the unique identifiers id_i to retrieve the matching raw data from local storage. MESS can also be integrated with other PIR schemes [19] to retrieve raw data from the server without revealing which data items are being retrieved.

2.2 Threat Model

We consider a semi-honest server that follows the protocol but wants to learn information about the data embeddings and about the queries. It can infer any geometric and semantic information directly from the perturbed codes [16]. The server knows the LSH mapping. It can record and analyze the complete view of the outsourced index and query execution. Specifically, during the offline phase, it can observe the encrypted data, the perturbed codes, shard assignment, and the HNSW graph structures. During the online

Tab. 1: Comparison of private keyword and semantic search solutions.

Scheme	Data Setting	Search Scheme	Cryptographic Algorithm	Search Method	Privacy			Accuracy	Efficiency
					Stored Data	Access Pattern	Search Pattern	Near-Plaintext Quality	Efficient Online Search
CGKO06 [14]	Encrypted DB	Keyword	SSE	Index-Based	✓	✗	✗	–	–
CLRZ18 [10]	Encrypted DB	Keyword	SSE, DP	Index-Based	✓	✓	✗	–	–
SOPK21 [40]	Encrypted DB	Keyword	SSE, DP	Index-Based	✓	✓	✓	–	–
LZXL25 [32]	Encrypted DB	Embedding	DCPE, DCE	Graph-Based	✓	✗	✗	✓	✓
PANTHER [29]	Encrypted DB	Embedding	PIR, HE, SS, GC	k-means	✓	✓	✓	✓	✗
Compass [51]	Encrypted DB	Embedding	ORAM	Graph-Based	✓	✓	–	✓	✗
Tiptoe [18]	Server-Held DB	Embedding	HE, PIR	k-means	–	✓	–	✓	✗
Wally [7]	Server-Held DB	Embedding	HE, PIR, DP	k-means	–	✓	–	✓	✗
PACMANN [50]	Server-Held DB	Embedding	PIR	Graph-Based	–	✓	–	✓	✗
Ours	Encrypted DB	Embedding	LSHRR	LSH, Graph-Based	✓	✓	✓	✓	✓

Abbreviations: DB: database; SSE: searchable symmetric encryption; DP: differential privacy; DCPE: distance-comparison-preserving encryption; DCE: distance comparison encryption; PIR: private information retrieval; HE: homomorphic encryption; SS: secret sharing; GC: garbled circuit; ORAM: oblivious random access memory; LSH: locality-sensitive hashing; and LSHRR: LSH randomized response.
Notes: Encrypted DB denotes data encrypted before outsourcing, while Server-Held DB denotes data owned by the server. ✓indicates that the goal is supported; ✗indicates that it is not protected or achieved; and "–" denotes an inapplicable or unreported guarantee. Near-plaintext quality means that the reported retrieval quality approaches the corresponding plaintext ANN baseline. Efficient online search means indexed, one-round retrieval without HE-based similarity evaluation or ORAM-protected graph traversal.

phase, it has complete visibility into query execution, including graph traversal traces, candidate sets, response sizes, and repeated ciphertexts. We assume that cryptographic primitives are secure, and that the server does not have access to client-side states.

2.3 Goals

MESS aims to achieve three properties: privacy, accuracy, and efficiency. We further break down privacy into stored-data, access-pattern, and search-pattern privacy. We formally analyze privacy in subsection 5.2, and provide a detailed evaluation of the other two in Sec. 6.

- ***Stored-data privacy.*** The server should not learn the contents of the original data tuple, consisting of the identifier or embedding, from the outsourced representation. Encrypted payloads should hide their plaintext contents, while the index incurs only well-defined leakage about the underlying embeddings.
- ***Access-pattern privacy.*** The server's ability to distinguish two query embeddings based on their execution traces is bounded. In other words, the query execution leakage is bounded.
- ***Search-pattern privacy.*** Across multiple queries, the server should have limited ability to determine whether different queries originate from the same query embedding value. In particular, it cannot identify repeated queries. Otherwise, it can group repeated submissions, observe their frequency and timing, and use auxiliary information to infer query values [9, 36].
- ***Accuracy.*** The system enables high-recall similar search, despite LSH encoding and randomized perturbation. In other words, the search results are close to those of non-private ANN search.
- ***Efficiency.*** The search incurs small computation and communication overhead, resulting in end-to-end latency in practice. In other words, the search is performed without scanning the full datasets, without overhead of encrypted computation, and without multi-round communications.

Discussion. Access-pattern privacy concerns leakage from a single execution, whereas search-pattern privacy concerns leakage across multiple executions. These two properties together provide a similar guarantee to query privacy in other private information retrieval (PIR) systems [5, 19].

3 Preliminaries

We use calligraphic uppercase letters for sets, spaces, and randomized mechanisms. Bold lowercase and uppercase letters denote vectors and matrices, respectively. Scalars are written in italic font. Let $\mathcal{X} \subseteq \mathbb{R}^d$ denote the original embedding space and $\mathcal{D} = \{(\mathrm{id}_i, \boldsymbol{x}_i)\}_{i=1}^N$ the original data, where $\boldsymbol{x}_i \in \mathcal{X}$. A query embedding is denoted by $\boldsymbol{q} \in \mathcal{X}$. We use $\|\boldsymbol{x}\|_2$ for the Euclidean norm and $\langle \cdot, \cdot \rangle$ for the inner product. $\theta_{\boldsymbol{x},\boldsymbol{x}'}$ denotes the angle between two non-zero embeddings, and $d_\theta(\boldsymbol{x}, \boldsymbol{x}') = \theta_{\boldsymbol{x},\boldsymbol{x}'}/\pi$ denotes their normalized angular distance.

$\mathcal{H}$ represents a LSH function family, and $h \in \mathcal{H}$ represents one function. Each graph index shard s uses a fixed κ-bit mapping $H_s : \mathcal{X} \to \mathcal{V}$, where $\mathcal{V} = \{0,1\}^\kappa$ is the binary embedding space. We write $\boldsymbol{v}_{i,s} = H_s(\boldsymbol{x}_i)$ and $\boldsymbol{v}_q^{(s)} = H_s(\boldsymbol{q})$ for the binary embeddings of a data item and query. A perturbed binary embedding is denoted by $\widetilde{\boldsymbol{v}}$, and $d_H(\cdot, \cdot)$ denotes the Hamming distance.

3.1 Locality-Sensitive Hashing under Angular Distance

LSH [22] maps high-dimensional vectors into compact binary codes such that vectors close to each other are more likely to share hash values. In semantic search, vector similarity is commonly measured by cosine similarity: $\mathrm{sim}_{\cos}(\boldsymbol{x}, \boldsymbol{x}') = \frac{\langle \boldsymbol{x}, \boldsymbol{x}' \rangle}{\|\boldsymbol{x}\|_2 \|\boldsymbol{x}'\|_2}$.

Isotropic Hashing (IsoHash). IsoHash [27] overcomes the problem of imbalanced or redundant hash bits when the vector distribution is anisotropic by learning an orthogonal transformation. In MESS, IsoHash is trained separately for each graph shard. The learned parameters are fixed and reused for both index construction

and query generation. We define IsoHash formally as $\Pi_{\text{IsoHash}} = (\text{PCA}, \text{Rotate}, \text{Hash})$.

- $(\mathbf{W}, \Lambda) \leftarrow \Pi_{\text{IsoHash}}.\text{PCA}(\mathbf{X} \in \mathbb{R}^{d\times n}, \kappa)$: Randomly select n samples to form the mean-centered training matrix $\mathbf{X}$, where κ denotes the hash length of each graph shard. PCA returns the projection matrix $\mathbf{W} \in \mathbb{R}^{d\times\kappa}$, formed by the eigenvectors corresponding to the largest κ eigenvalues, and the covariance matrix of the projected dimensions: $\Lambda = \mathbf{W}^T\mathbf{X}\mathbf{X}^T\mathbf{W} = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_\kappa)$.
- $\mathbf{Q} \leftarrow \Pi_{\text{IsoHash}}.\text{Rotate}(\Lambda \in \mathbb{R}^{\kappa\times\kappa})$: Given the covariance matrix of the projected dimensions Λ, output an isotropic orthogonal rotation matrix $\mathbf{Q} \in \mathbb{R}^{\kappa\times\kappa}$ satisfying $\mathbf{Q}^T\mathbf{Q} = I$, such that the diagonal elements of $\mathbf{Q}^T\Lambda\mathbf{Q}$ are all equal, i.e., $[\mathbf{Q}^T\Lambda\mathbf{Q}]_{11} = [\mathbf{Q}^T\Lambda\mathbf{Q}]_{22} = \cdots = [\mathbf{Q}^T\Lambda\mathbf{Q}]_{\kappa\kappa} = \bar{\lambda}$. Since an orthogonal transformation preserves the trace of a matrix, the target mean variance is necessarily $\bar{\lambda} = \frac{1}{\kappa}\sum_{i=1}^{\kappa}\lambda_i$. This optimization is typically solved by Lift-and-Projection (LP) or Gradient Flow (GF) algorithms.
- $h \leftarrow \Pi_{\text{IsoHash}}.\text{Hash}(\boldsymbol{x} \in \mathbb{R}^d, \mathbf{W}, \mathbf{Q})$: Given an input data point $\boldsymbol{x}$ together with matrices $\mathbf{W}$ and $\mathbf{Q}$, output the final κ-bit binary embedding h, defined by $h = \text{sgn}(\mathbf{Q}^T\mathbf{W}^T\boldsymbol{x})$.

3.2 LSHRR-Based Embedding Perturbation

Traditional ϵ-differential privacy defines neighboring inputs through a binary adjacency relation. This is less suitable for semantic search, where the protected inputs are continuous embeddings and privacy loss should depend on their distance. We therefore use extended differential privacy (XDP), which is based on the distance induced by a fixed binary hash mapping. Each graph shard learns an independent hash mapping. Let $H : \mathcal{X} \to \{0,1\}^\kappa$, where $H(\boldsymbol{x}) = \big(h_1(\boldsymbol{x}), \ldots, h_\kappa(\boldsymbol{x})\big)$. For two embeddings $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{X}$, we define the induced Hamming pseudometric as $d_H(\boldsymbol{x}, \boldsymbol{x}') = \sum_{i=1}^{\kappa}\mathbf{1}[h_i(\boldsymbol{x}) \neq h_i(\boldsymbol{x}')]$. LSHRR [16] applies randomized response independently to each bit of $H(\boldsymbol{x})$. Given a per-bit privacy parameter ϵ, the bit-flip probability is $p = 1/(e^\epsilon + 1)$. The resulting mechanism Q_H defines a distribution over $\{0,1\}^\kappa$. For any output $\boldsymbol{y} \in \{0,1\}^\kappa$, $\Pr[Q_H(\boldsymbol{x}) = \boldsymbol{y}] = \prod_{i=1}^{\kappa} p^{|y_i - h_i(\boldsymbol{x})|}(1-p)^{1-|y_i-h_i(\boldsymbol{x})|}$. It follows that bitwise randomized response under a fixed binary hash mapping satisfies, for every $\boldsymbol{x}, \boldsymbol{x}' \in \mathcal{X}$ and $S \subseteq \{0,1\}^\kappa$, $\Pr[Q_H(\boldsymbol{x}) \in S] \leq e^{\epsilon d_H(\boldsymbol{x},\boldsymbol{x}')}\Pr[Q_H(\boldsymbol{x}') \in S]$. In other words, Q_H satisfies $(\epsilon d_H, 0)$-XDP under the fixed hash-induced Hamming pseudometric. As a result, embeddings that differ in a few fixed-hash coordinates induce similar output distributions.

3.3 Hierarchical Navigable Small World

The Hierarchical Navigable Small World (HNSW) [34] is a widely used vector index that achieves high search recall. It combines skip lists with navigable proximity graphs. Let $\mathcal{G} = \{G_0, G_1, \ldots, G_L\}$ denote the $L+1$ graph layers. The graph construction is guided by four parameters: M_{hnsw} that limits the number of connections per node, with the bottom layer typically allowing about $2M_{\text{hnsw}}$ connections; $ef_{\text{construction}}$ that controls the candidate search during index construction; ef_{search} that determines the recall–latency trade-off during search; and m_L that controls the exponentially decreasing node distribution across layers, typically set to $1/\ln(M_{\text{hnsw}})$.

The core operations are defined as follows.

- $(\mathcal{G}, ep_L) \leftarrow \Pi_{\text{HNSW}}.\text{InitGraph}()$: Initialize an empty hierarchical graph, its layers, and the entry point used by insertion and search.
- $\mathcal{G}' \leftarrow \Pi_{\text{HNSW}}.\text{Insert}(\mathcal{G}, \boldsymbol{v} \in \mathcal{V})$: Insert $\boldsymbol{v}$ and output the updated graph $\mathcal{G}'$. The algorithm samples its maximum level, greedily routes through the upper layers, searches local candidates using $ef_{\text{construction}}$, selects neighbors bounded by M_{hnsw} (approximately $2M_{\text{hnsw}}$ at the bottom layer), and creates bidirectional edges.
- $\mathcal{R} \leftarrow \Pi_{\text{HNSW}}.\text{Search}(\mathcal{G}, \boldsymbol{v}_q \in \mathcal{V}, K)$: Given a query $\boldsymbol{v}_q$, the algorithm greedily starts from the top layer, searches the bottom layer with candidate budget ef_{search}, and returns the approximate top-K nearest-neighbor set $\mathcal{R}$.

4 Design of MESS

In this section, we describe the design of MESS, which achieves privacy, accuracy, and efficiency. MESS adopts differential privacy, allowing it to bound the privacy leakage and avoid the overheads of other cryptographic approaches. However, searching over perturbed codes degrades accuracy because the distances between codes do not match those between the original embeddings. The system can improve accuracy by returning larger candidate sets to the client, but doing so increases communication and computational overhead. Below, we elaborate on the two challenges related to accuracy and discuss our approach.

Challenge 1: Candidate-set expansion under perturbation. High bit-flip probabilities provide strong privacy, but distort nearest-neighbor rankings. Our experiments confirm this effect: with a single perturbed graph, SIFT achieves only 56.65% recall using 1,500 candidates, while LAION achieves only 69.6% recall using 700 candidates. To improve recall, the server can return a larger candidate set for client-side reranking, but this incurs higher communication and computational overhead. Reducing the bit-flip probability could also improve recall, but the codes expose more semantic information, thereby reducing privacy.

Our approach. MESS employs multiple index graphs (or shards) to reduce competition among candidates within each shard. Randomized response may cause non-target points to overtake a true neighbor in Hamming space. Let $N(d)$ be the number of points at distance d, and let $P_{\text{overtake}}(d)$ be the probability that such a point overtakes the true neighbor after perturbation. The expected overtaking count in a single graph is $\mu_{\text{single}} = \sum_{d=0}^{\kappa} N(d)P_{\text{overtake}}(d)$. When the codes are replicated uniformly in t out of M shards, $\mu_{\text{shard}} = (t/M)\mu_{\text{single}}$. Each shard uses a small candidate set, and aggregation over all the shards gives the true neighbors multiple opportunities to be recovered.

Challenge 2: Search instability under perturbation. Random bit flipping may severely displace the true neighbors of certain embeddings. A single graph may perform well for most queries, but require a prohibitively large candidate set for queries with severely displaced neighbors.

Our approach. MESS builds multiple shards, each with an independent IsoHash mapping and independently sampled randomized-response coins. A true neighbor that is poorly positioned in one shard may remain close to the query in another. Let $P_{\text{hit}}(b)$ be the recovery probability in one shard with local candidate budget b, under an approximate independence assumption, the recovery probability from at least one of the t shards is $P_{\text{multi}}(b) \approx 1 - (1 - P_{\text{hit}}(b))^t$.

4.1 Multi-Graph Construction

Let $\mathcal{D} = \{(\mathrm{id}_i, \boldsymbol{x}_i)\}_{i=1}^{N} \subseteq \mathcal{X}$ denote the original embeddings. The system maintains M logically separated HNSW graph shards, denoted by $\mathbb{G} = \{\mathcal{G}^{(1)}, \ldots, \mathcal{G}^{(M)}\}$. Each shard $\mathcal{G}^{(j)}$ stores only the perturbed index binary embeddings (or perturbed codes) and the associated encrypted payloads.

Fig. 2 illustrate the multi-graph construction. For each embedding i, the client assigns it to a set of shards $\mathcal{S}_i \leftarrow \mathsf{Route}(i, M, t)$. We instantiate Route as a data-independent pseudorandom function that permutes $[M]$ and selects the first t shards. For every shard $j \in \mathcal{S}_i$, the client computes the shard-level binary embedding $\boldsymbol{v}_{i,j} \leftarrow \Pi_{\mathrm{IsoHash}}.\mathsf{Hash}(\boldsymbol{x}_i, \boldsymbol{\mu}_j, \mathbf{W}_j, \mathbf{Q}_j)$, then applies LSHRR to obtain $\widetilde{\boldsymbol{v}}_{i,j} \leftarrow \Pi_{\mathrm{LSHRR}}(\boldsymbol{v}_{i,j}; p)$. It encrypts the payload $m_i = \mathrm{id}_i \| \boldsymbol{x}_i$ with the shard key to obtain $\mathrm{ct}_{i,j} \leftarrow \Pi_{\mathrm{Enc}}.\mathrm{Enc}_{sk_j}(m_i)$. The client also generates a shard-local label $\ell_{i,j}$, used later for deletion. The resulting tuple $(\widetilde{\boldsymbol{v}}_{i,j}, \ell_{i,j}, \mathrm{ct}_{i,j})$ is inserted into $\mathcal{G}^{(j)}$.

In an ideal setting, the M graph shards are maintained by M non-colluding servers. In this case, an attacker controlling a single server can observe a given embedding with probability t/M. Its local view can therefore exhibit a subsampling effect under the standard conditions for differential-privacy amplification. MESS assumes a single-server setting, in which the attacker has the complete view of all the embeddings. Therefore, we do not use t/M as a subsampling probability in the privacy analysis.

4.2 Single-Server Multi-Graph

The server maintains all graph shards. MESS employs several mechanisms to reduce cross-shard linkage.

- *Shard-specific projection functions.* Each shard uses different IsoHash parameters. The same original embedding $\boldsymbol{x}_i$ is therefore mapped to different shard-level binary embeddings, making bit coordinates across different shards not directly comparable.
- *Independent perturbation.* Randomized response uses fresh coins for every shard. Even when the same embedding appears in multiple shards, its binary embeddings are independently perturbed, preventing direct equality comparison and reducing the effectiveness of averaging attacks.
- *Randomized shard assignment.* Each embedding is assigned to only t pseudorandomly selected shards instead of all M shards. This limits the number of server-visible entries associated with the same item and avoids fixed shard co-occurrence patterns.
- *Shard-level labels and encryption keys.* Labels are generated independently for different shards, while payloads are encrypted with shard-level keys and fresh randomness. As a result, identifier or ciphertext equality cannot be used to link embeddings across different shards.

These mechanisms make cross-shard linkage more difficult for the attacker, but do not eliminate it. Our formal analysis (Sec. 5) accounts for all t perturbed codes originating from the same embedding. We also evaluate representative cross-shard attacks to measure whether the server can link embeddings across shards.

4.3 Processing User Queries

Repeated queries may reveal the underlying value if each query is independently perturbed. By collecting many versions of the same query, the attacker can average out the randomness and infer the original binary embedding. To mitigate this averaging attack, MESS adopts a two-phase LSHRR mechanism, inspired by the memoization technique proposed by RAPPOR [15]. Fig. 3 illustrates the query workflow.

Permanent LSHRR. For a query $\boldsymbol{q}$, let $\boldsymbol{v}_{\boldsymbol{q}}$ denote its unperturbed binary embedding, and let p_{PRR} denote the permanent flip probability. In this phase, the client first checks its local cache. If $\boldsymbol{q}$ is issued

Algorithmic Primitives: Isotropic hashing Π_{IsoHash}, locality-sensitive hashing randomized response Π_{LSHRR}, symmetric encryption Π_{Enc}, HNSW indexing Π_{HNSW}.
Input: Plaintext embedding database $\mathcal{D} = \{(\mathrm{id}_i, \boldsymbol{x}_i)\}_{i=1}^{N}$ with $\boldsymbol{x}_i \in \mathcal{X}$, shard-level binary embedding length κ, total number of graph shards M, routing multiplicity t, randomized-response flip probability p, and HNSW parameters.
Output: Public configuration pp, client-side state st_C, and server-side multi-graph index $\mathrm{st}_{\mathcal{S}}$.

[Phase 1: Parameter Setup]

1: The client samples representative training data from $\mathcal{D}$ and trains shard-specific IsoHash parameters $\mathrm{st}_{\mathrm{hash}} = \{(\boldsymbol{\mu}_j, \mathbf{W}_j, \mathbf{Q}_j)\}_{j=1}^{M}$. The public configuration is set as $\mathrm{pp} = (\kappa, M, t, p, \mathrm{params}_{\mathrm{HNSW}})$.

[Phase 2: Multi-Graph Initialization]

2: The client prepares M independent symmetric encryption keys and forms the shard key set $\mathcal{K} = \{sk_1, \ldots, sk_M\}$.

3: The cloud server initializes M logically separated HNSW graph shards. The outsourced multi-graph index is denoted by $\mathbb{G} = \{\mathcal{G}^{(1)}, \ldots, \mathcal{G}^{(M)}\}$, where each shard $\mathcal{G}^{(j)} = \{G_0^{(j)}, \ldots, G_{L_j}^{(j)}\}$ is an independent multi-layer HNSW index.

[Phase 3: Perturbation and Insertion]

4: For each data point $(id_i, \boldsymbol{x}_i)$, the client obtains its selected shard set by calling $S_i \leftarrow Route(id_i, M, t)$.

5: For each selected shard $j \in \mathcal{S}_i$, the client computes the shard-level binary embedding $\boldsymbol{v}_{i,j} \leftarrow \Pi_{\mathrm{IsoHash}}.\mathrm{Hash}(\boldsymbol{x}_i, \boldsymbol{\mu}_j, \mathbf{W}_j, \mathbf{Q}_j)$, and perturbs it as $\widetilde{\boldsymbol{v}}_{i,j} \leftarrow \Pi_{\mathrm{LSHRR}}(\boldsymbol{v}_{i,j}; p)$.

6: The client generates a shard-local label $\ell_{i,j}$ and forms the encrypted payload $m_i \leftarrow \mathrm{id}_i \| \boldsymbol{x}_i$, where id_i is used for duplicate removal and result recovery, and $\boldsymbol{x}_i$ is used for client-side exact reranking. The client encrypts m_i under sk_j with fresh encryption randomness, yielding $\mathrm{ct}_{i,j} \leftarrow \Pi_{\mathrm{Enc}}.\mathrm{Enc}_{sk_j}(m_i)$.

7: The tuple $(\widetilde{\boldsymbol{v}}_{i,j}, \ell_{i,j}, \mathrm{ct}_{i,j})$ is inserted into the HNSW graph shard $\mathcal{G}^{(j)}$ by running $\mathcal{G}^{(j)} \leftarrow \Pi_{\mathrm{HNSW}}.\mathrm{Insert}(\mathcal{G}^{(j)}, \widetilde{\boldsymbol{v}}_{i,j}, \ell_{i,j}, \mathrm{ct}_{i,j})$.

[Phase 4: State Finalization]

8: The protocol outputs $\mathrm{st}_C = (\mathrm{st}_{\mathrm{hash}}, \mathcal{K})$ and $\mathrm{st}_{\mathcal{S}} = \mathbb{G}$.

Fig. 2: Multi-graph index construction protocol ($\Pi_{\mathsf{GraphBuild}}$).

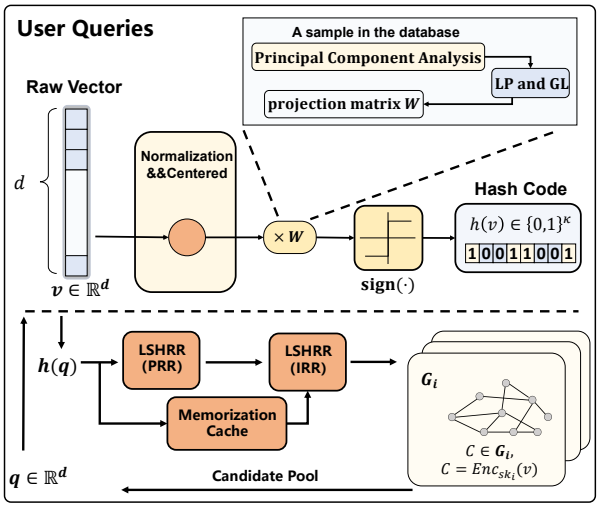


Fig. 3: Systematic illustration of the query flow.

for the first time, the client samples

$$\widetilde{v}_q^{(\mathrm{PRR})} \leftarrow \Pi_{\mathrm{LSHRR}}(v_q, p_{\mathrm{PRR}}),$$

and writes $\widetilde{v}_q^{(\mathrm{PRR})}$ to the cache. If q is in the cache, $\widetilde{v}_q^{(\mathrm{PRR})}$ is reused. As a result, repeated submissions do not have independent perturbations of the original query binary embedding.

Instantaneous LSHRR. Before sending the query, the client applies an instantaneous perturbation

$$\widetilde{v}_q^{(\mathrm{IRR})} \leftarrow \Pi_{\mathrm{LSHRR}}\big(\widetilde{v}_q^{(\mathrm{PRR})}, p_{\mathrm{IRR}}\big),$$

where p_{IRR} is the instantaneous flip probability. The server uses $\widetilde{v}_q^{(\mathrm{IRR})}$ as input to the search. Since $\widetilde{v}_q^{(\mathrm{IRR})}$ varies across repeated queries, the server cannot link them.

4.4 MESS Protocol

We now combine the graph construction and query protocol above into the complete protocol. MESS consists of four key operations: initialization, secure query generation, search, and decryption and reranking. It also supports update operations, i.e., insertion and deletion.

(1) *Initialization.* Given $\mathcal{D} = \{(\mathrm{id}_i, \boldsymbol{x}_i)\}_{i=1}^N$, the client trains shard-specific hashing parameters, generates shard keys, and initializes $\mathbb{G} = \{\mathcal{G}^{(1)}, \ldots, \mathcal{G}^{(M)}\}$. Each item is assigned to a set of shards $\mathcal{S}_i \leftarrow \mathrm{Route}(\mathrm{id}_i, M, t)$. For every $j \in \mathcal{S}_i$, the client computes and perturbs the shard-level binary embedding, encrypts $m_i = \mathrm{id}_i \| \boldsymbol{x}_i$ under sk_j, and sends the resulting values to the server. The server inserts these values into the corresponding HNSW shards. The resulting states are $\mathrm{st}_C = (\mathrm{st}_{\mathrm{hash}}, \mathcal{K})$ and $\mathrm{st}_{\mathcal{S}} = \mathbb{G}$.

(2) *Secure query generation.* Given a query embedding q, the client derives a query binary embedding for each shard. It applies permanent perturbation to prevent averaging across repeated submissions, followed by fresh instantaneous perturbation for each query. The resulting perturbed query embeddings are sent to the server.

(3) *Search.* The server searches the HNSW shards using Hamming distance between the perturbed query and index nodes. It aggregates the shard-level candidate sets and returns their encrypted payloads.

(4) *Decryption and reranking.* The client decrypts the returned payloads, removes duplicates using the identifiers, and reranks the remaining candidates using the original query and data embeddings. It then outputs the final top-K results.

(5) *Insertion.* To insert a new tuple $(\mathrm{id}_i, \boldsymbol{x}_i)$, the client reuses the existing hashing parameters and shard keys. It selects $\mathcal{S}_i$, generates independently perturbed binary embeddings and encrypted payloads for the selected shards, and computes the shard-local label $\ell_{i,j}$. The server inserts the entries into the corresponding HNSW shards, while the client stores $\{(j, \ell_{i,j})\}_{j \in \mathcal{S}_i}$ for future deletion. Insertion does not require rebuilding the index.

(6) *Deletion.* The client retrieves the shard-local labels associated with id_i and sends deletion requests to the server. The server marks the corresponding index nodes as inactive and excludes them from the search. The server performs periodic index rebuilding to remove inactive nodes.

5 Theoretical Analysis

5.1 Correctness Analysis

We characterize how perturbation affects shard-level ranks and candidate recovery.

Proposition 1 (Approximate Candidate Recovery). Consider a target item in shard s at clean Hamming distance d_s^* from the query, and let $N_s(d)$ be the number of background items at distance d. The effective query flip probability is $p_q = p_{\mathrm{PRR}}(1-p_{\mathrm{IRR}})+p_{\mathrm{IRR}}(1-p_{\mathrm{PRR}})$, and its combination with the storage-side perturbation is $p_\oplus = p_D(1-p_q) + p_q(1-p_D)$. A background item at distance d overtakes the target with approximate probability

$$P_{\mathrm{overtake},s}(d) \approx \Phi\left(\frac{(d_s^* - d)(1-2p_\oplus)}{\sqrt{2\kappa p_\oplus(1-p_\oplus)}}\right).$$

Under independent overtaking events, their count is Poisson binomial with mean $\mu_s = \sum_d N_s(d) P_{\mathrm{overtake},s}(d)$ and variance $\sigma_s^2 = \sum_d N_s(d) P_{\mathrm{overtake},s}(d)(1 - P_{\mathrm{overtake},s}(d))$. When sufficiently many items contribute, the target rank satisfies $Rank_s \approx \mathcal{N}(1+\mu_s, \sigma_s^2)$. For shard-level candidate budget P, its inclusion probability is

$$P_{\mathrm{hit},s}(P) \approx \Phi\left(\frac{P + \frac{1}{2} - (1+\mu_s)}{\sigma_s}\right).$$

Across conditionally independent selected shards, the aggregate recovery probability is approximately $1 - \prod_{s \in \mathcal{S}_i}(1 - P_{\mathrm{hit},s}(P))$. If the aggregate candidate set contains the true Top-K items, client-side exact reranking returns the plaintext Top-K result.

Proof Sketch. Among the d coordinates where the clean embeddings differ, the perturbed bits remain different with probability $1 - p_\oplus$, so their mismatch count follows $X_d \sim \mathrm{Binomial}(d, 1-p_\oplus)$. Among the remaining $\kappa - d$ coordinates, perturbation creates a mismatch with probability $p_\oplus$, giving $Y_d \sim \mathrm{Binomial}(\kappa - d, p_\oplus)$. Independence across bits gives $\widetilde{D}_d = X_d + Y_d$, with mean $\kappa p_\oplus + d(1-2p_\oplus)$ and variance $\kappa p_\oplus(1-p_\oplus)$.

Under the standard independence approximation, the difference between the perturbed distances of a background item and the target has mean $(d - d_s^*)(1 - 2p_\oplus)$ and variance $2\kappa p_\oplus(1 - p_\oplus)$. Its Gaussian approximation gives $P_{\text{overtake},s}(d)$.

For each background item i at distance d, let $I_{i,d} = \mathbf{1}\{\widetilde{D}_{i,d} < \widetilde{D}_{d_s^*}\}$. Then $Rank_s = 1 + \sum_d \sum_{i=1}^{N_s(d)} I_{i,d}$. The indicators have distance-dependent Bernoulli parameters, so their sum is Poisson binomial with the stated mean and variance. Its normal approximation, with the $1/2$ continuity correction, gives $P_{\text{hit},s}(P)$. Conditional independence across shards gives the aggregate recovery probability. Finally, exact distance computation and reranking return the plaintext Top-K result whenever all true Top-K items are present. □

This approximation captures the effects of perturbation, candidate budget, and multi-shard recovery. The detailed derivation of the shard-level rank distribution and its connection to Recall and MRR are provided in Appendix B.1.

5.2 Security and Privacy of the Server View

The complete server view contains all graph shards, perturbed index binary embeddings, shard-local HNSW structures, perturbed query reports, and search traces. We protect stored embedding values, submitted query values, and the reuse relation among repeated queries, corresponding to stored-data, access-pattern, and search-pattern privacy. The term *complete server view* specifies the adversarial observation rather than a new privacy definition.

Our security statement is hybrid. Differential privacy protects the randomized index and query reports together with information derived from them, while randomized authenticated encryption protects payload contents. These guarantees do not depend on the success of a concrete inference method.

5.2.1 Scope and Fixed ISO-LSH Mappings. For stored data, we use substitution adjacency: two databases contain the same identifiers and differ only in the embedding associated with one identifier. Thus, the guarantee protects the embedding value rather than the presence of its identifier. Each shard G_s uses a pretrained mapping $H_s : \mathcal{X} \to \{0,1\}^{\kappa_s}$ fixed before the protected database is instantiated. The mappings may be correlated; the proofs require fresh randomized-response coins for each record, shard, and coordinate. If ISO-LSH is trained on the protected database, its training requires a separate privacy analysis. We therefore assume that the training data are public, disjoint from the protected database, or excluded from the neighboring relation.

A mechanism $\mathcal{M}$ satisfies (ξ, δ)-XDP if, for every pair x, x' and measurable event E,

$$\Pr[\mathcal{M}(x) \in E] \le e^{\xi(x,x')} \Pr[\mathcal{M}(x') \in E] + \delta(x, x').$$

Our guarantee uses the Hamming pseudometric induced by the fixed ISO-LSH mappings. It differs from angular-XDP averaged over freshly sampled random-LSH functions [16]; the collision distribution from that sampling is not applied to the pretrained mappings.

5.2.2 Stored Multi-Graph Index. Let $p_D \in (0, 1/2)$ be the storage-side bit-flip probability; define $\eta_D = \ln((1 - p_D)/p_D)$, and let $d_{H_s}(\boldsymbol{x}, \boldsymbol{x}') = d_H(H_s(\boldsymbol{x}), H_s(\boldsymbol{x}'))$.

Proposition 2 (Single-Shard XDP). Conditioned on a record being assigned to G_s, bitwise randomized response satisfies η_D-XDP with respect to d_{H_s}. A fixed pair $q = (\boldsymbol{x}_0, \boldsymbol{x}_1)$ therefore has shard-level parameter $\varepsilon_{s,q} = \eta_D d_s(q)$, where $d_s(q) = d_H(H_s(\boldsymbol{x}_0), H_s(\boldsymbol{x}_1))$. Thus, if two embeddings differ in d fixed-hash coordinates, the shard-local observation differs by at most a factor of $e^{\eta_D d}$. A smaller hash distance results in stronger privacy.

Sketched Proof. Equal clean bits induce identical output distributions. For a differing bit, the likelihood ratio is at most $(1 - p_D)/p_D = e^{\eta_D}$. Independence across coordinates gives

$$\frac{\Pr[Y_s(\boldsymbol{x}) = \boldsymbol{y}]}{\Pr[Y_s(\boldsymbol{x}') = \boldsymbol{y}]} \le e^{\eta_D d_{H_s}(\boldsymbol{x}, \boldsymbol{x}')}.$$

Summing over outputs in E proves the claim; exchanging $\boldsymbol{x}$ and $\boldsymbol{x}'$ proves the reverse direction. □

Each record is assigned to exactly t of the M shards independently of its embedding. Let $\mathfrak{R}$ contain the t-shard sets that may be selected; for identifier-fixed assignment, it contains only the realized set. Define $D_D(\boldsymbol{x}, \boldsymbol{x}') = \max_{\mathcal{R} \in \mathfrak{R}} \sum_{s \in \mathcal{R}} d_{H_s}(\boldsymbol{x}, \boldsymbol{x}')$.

Theorem 5.1 (Complete Stored-Index Privacy). *For every measurable event E,*

$$\Pr[\mathsf{V}_D(\boldsymbol{x}) \in E] \le e^{\eta_D D_D(\boldsymbol{x}, \boldsymbol{x}')} \Pr[\mathsf{V}_D(\boldsymbol{x}') \in E],$$

with the symmetric inequality obtained by exchanging $\boldsymbol{x}$ and $\boldsymbol{x}'$. Hence, the stored multi-graph index is η_D-XDP with respect to D_D. If every shard has κ bits, it also satisfies $(t\kappa\eta_D, 0)$-DP under unrestricted substitution adjacency. In other words, for two stored embeddings $\boldsymbol{x}$ and $\boldsymbol{x}'$, the complete multi-graph observation differs by at most a factor of $e^{\eta_D D_D(\boldsymbol{x}, \boldsymbol{x}')}$, where D_D aggregates the differing hash coordinates across the selected shards.

Sketched Proof. Condition on a selected set $\mathcal{R}$. Only the t corresponding shard-local entries depend on the substituted embedding. Proposition 2 and composition give $\eta_D \sum_{s \in \mathcal{R}} d_{H_s}(\boldsymbol{x}, \boldsymbol{x}')$. For randomized assignment, its distribution is identical in both worlds; averaging the conditional inequalities and bounding each sum by D_D proves the result. The fixed-assignment case follows directly.

Each HNSW shard is constructed from perturbed codes, public parameters, and world-independent randomness. Its topology and subsequent inference outputs are therefore post-processing and incur no additional privacy cost. □

The theorem conservatively grants the server the selected shard set and perfect association of the changed record's shard-local entries. Subsampling amplification is not applied to the complete view: although one prespecified shard contains the record with probability t/M, the complete server observes exactly t releases.

Pair-Specific XDP Accounting. For pair $q = (\boldsymbol{x}_0, \boldsymbol{x}_1)$ and selected set $\mathcal{R}$, let $u_{\mathcal{R}}(q) = \sum_{s \in \mathcal{R}} d_s(q)$. Its route-conditioned pure-XDP value is $\xi_{\mathcal{R}}(q) = \eta_D u_{\mathcal{R}}(q)$. For randomized assignment, define $w_q(u) = \Pr_{\mathcal{R}}[u_{\mathcal{R}}(q) = u]$; for fixed assignment, w_q is a point mass.

Conditioned on u differing coordinates, $Z \sim \text{Binomial}(u, 1 - p_D)$ and $L_u = (2Z - u)\eta_D$. The corresponding privacy profile is

$$\delta_u^{\text{RR}}(\varepsilon) = \sum_{z=0}^{u} \binom{u}{z} (1 - p_D)^z p_D^{u-z} \left[1 - e^{\varepsilon - (2z-u)\eta_D}\right]_+.$$

Corollary 1 (Pair-Specific XDP Profile and Upper Tail). For target δ_0, the evaluated pair has approximate-XDP value $\xi^{\mathrm{PLD}}_{D,q}(\delta_0) = \inf\{\varepsilon : \sum_u w_q(u)\delta^{\mathrm{RR}}_u(\varepsilon) \le \delta_0\}$. Moreover,

$$\Pr[L^{\mathrm{impl}}_{D,q} > \varepsilon] \le \min\left\{1, \inf_{0\le\alpha<\varepsilon} \frac{\delta^{\mathrm{PLD}}_{D,q}(\alpha)}{1-e^{\alpha-\varepsilon}}\right\},$$

where $\delta^{\mathrm{PLD}}_{D,q}(\alpha) = \sum_u w_q(u)\delta^{\mathrm{RR}}_u(\alpha)$.

Skeched Proof. Consider an augmented release that also reveals the challenge record, its selected shards, and their ground-truth correspondence. Conditioned on u, the binomial expression is its exact hockey-stick profile. The assignment has the same distribution in both worlds, so averaging over w_q yields the augmented profile. Removing the additional correspondence information is post-processing and cannot increase hockey-stick divergence.

For $0 \le \alpha < \varepsilon$, $L > \varepsilon$ implies $[1-e^{\alpha-L}]_+ \ge 1-e^{\alpha-\varepsilon}$. Taking expectations and optimizing over α proves the tail bound; randomized-response symmetry gives the reverse direction. □

We evaluate $\xi^{\mathrm{PLD}}_{D,q}(\delta_0)$ using the actual shard-specific distances of real neighboring pairs. The maximum over a finite challenge set applies only to those pairs; Theorem 5.1 remains the unrestricted XDP guarantee.

Cross-Shard Linkage Evaluation. The formal analysis assumes perfect association of all t shard-local entries belonging to the changed record. Our experiments test how much of this XDP evidence can be organized using MDS, Topology, GSGM, and their cycle-consistent joint combination.

For method $\mathcal{A}$, let $C_{\mathcal{A}}(\mathsf{V})$ be its predicted cross-shard cluster and let $v^\star_s$ denote the target entry in G_s. Ground truth is used only after inference. The distance of correctly associated target entries is

$$U_{\mathcal{A},q}(\mathsf{V}) = \sum_{(s,v)\in C_{\mathcal{A}}(\mathsf{V})} \mathbf{1}[v = v^\star_s]\, d_s(q),$$

giving recovered pure-XDP evidence $\xi^{\mathrm{rec}}_{\mathcal{A},q} = \eta_D U_{\mathcal{A},q}$. Recovering all t target entries reaches the route-conditioned complete-view value, whereas retaining one anchor gives only its single-shard value. Failed and false matches contribute zero direct target distance.

For each challenge pair, we construct two neighboring indexes that differ only in one stored embedding. Each method receives the server-visible view and fixed auxiliary seeds, and outputs a predicted cross-shard cluster and a distinguishing score. Because the inferred cluster is data-dependent, we evaluate matching accuracy, AUC, distinguishing advantage, and empirical hockey-stick profiles. Inverting the profile yields an empirical pair-specific XDP value for each evaluated method without replacing the formal guarantee. Complete procedures, parameters, and results are provided in Appendix A.2.

Payload Confidentiality. Identifiers and original embeddings are stored in equal-length payloads using randomized authenticated encryption. Under IND-CPA security, fresh nonces and independent shard keys prevent ciphertext equality from directly associating entries of the same record across shards, assuming no common identifier or equivalent metadata is exposed.

5.2.3 Access-Pattern Privacy. For query-value privacy, the released index, client identity, submission schedule, PRR-reuse pattern, and search configuration are fixed, while a single logical query value is substituted. The server observes the perturbed query reports and complete HNSW access transcript. Our goal is to bound the query information revealed by these visible accesses, rather than to provide ORAM-style access-pattern obliviousness.

Let $a = p_{\mathrm{PRR}}$ and $b = p_{\mathrm{IRR}}$, where $a, b \in (0, 1/2)$. A permanent report is generated once for each logical-query coordinate and reused, whereas every submission applies fresh IRR coins. Randomness is independent across coordinates and shards. For one clean bit v submitted R times, let $n_v(\boldsymbol{y})$ be the number of reports in $\boldsymbol{y} = (y_1, \ldots, y_R)$ equal to v. The sequence probability and maximum symmetric log-likelihood ratio are

$$P^{(R)}_v(\boldsymbol{y}) = (1-a)(1-b)^{n_v} b^{R-n_v} + a b^{n_v}(1-b)^{R-n_v}$$

$$\eta_R = \max_{\boldsymbol{y}} \left|\ln \frac{P^{(R)}_v(\boldsymbol{y})}{P^{(R)}_{1-v}(\boldsymbol{y})}\right| = \ln \frac{(1-a)(1-b)^R + ab^R}{a(1-b)^R + (1-a)b^R}.$$

The maximum occurs when all R reports agree. Hence, η_R jointly accounts for the memoized PRR state and all IRR reports, rather than applying basic composition across submissions.

For a fixed contacted shard set $\mathcal{S}_Q$, define $D_Q(\boldsymbol{q}, \boldsymbol{q}') = \sum_{s\in\mathcal{S}_Q} d_H(H_s(\boldsymbol{q}), H_s(\boldsymbol{q}'))$. If the set is sampled independently of the clean query, the pure-XDP distance is the maximum of this sum over all possible contacted sets. Routing based on an unperturbed query is outside the theorem's scope.

Theorem 5.2 (Complete Access-Transcript Privacy). For every measurable event E,

$$\Pr[\mathsf{V}_{\mathrm{AP}}(\boldsymbol{q}) \in E] \le e^{\eta_R D_Q(\boldsymbol{q},\boldsymbol{q}')} \Pr[\mathsf{V}_{\mathrm{AP}}(\boldsymbol{q}') \in E],$$

with the reverse inequality obtained by exchanging $\boldsymbol{q}$ and $\boldsymbol{q}'$. Thus, the complete access transcript is η_R-XDP with respect to D_Q. Therefore, for any set of node-access traces, the probability of it being the trace of $\boldsymbol{q}$ is at most $e^{\eta_R D_Q(\boldsymbol{q},\boldsymbol{q}')}$ times the probability of it being the trace of $\boldsymbol{q}'$. Observing the HNSW traversal thus reveals no more about the query than the perturbed codes already do.

Skeched Proof. Equal clean bits induce identical sequence distributions, while each differing coordinate contributes at most η_R. Composition over the differing coordinates and contacted shards gives $\eta_R D_Q(\boldsymbol{q}, \boldsymbol{q}')$ for the complete query reports. Given the fixed index, HNSW receives only these reports. Visited nodes, traversal order, candidate evolution, returned handles, response sizes, and cross-shard correspondences inferred from the index and traces are therefore post-processing and incur no additional privacy cost. Since the reports are included in the server view and recoverable by projection, the complete access transcript and report mechanism have the same privacy profile. □

Accordingly, no separate access-pattern experiment is required: its XDP budget is computed directly as $\xi_{\mathrm{AP}}(\boldsymbol{q}, \boldsymbol{q}') = \eta_R D_Q(\boldsymbol{q}, \boldsymbol{q}')$. The guarantee requires every trace field to depend only on the perturbed reports, the fixed index, public parameters, and query-independent randomness. Clean-query routing, clean-distance calculations, stable query identifiers, or external side channels must be removed or analyzed separately.

5.2.4 Search-Pattern Privacy. Search-pattern adjacency compares workloads with the same number and timing of submissions, contacted shards, and search configuration, but different query-reuse relations. For $R \geq 2$, all submissions in $W_0 = (\boldsymbol{q}, \ldots, \boldsymbol{q})$ share one permanent report. In $W_1 = (\boldsymbol{q}, \ldots, \boldsymbol{q}, \boldsymbol{q}')$, the final submission belongs to a new logical query and uses an independently sampled permanent report.

For one coordinate, let $r_v(z)$ be the PRR probability of permanent bit z given clean bit v, and let $i_z(y)$ be the IRR probability of report y given z. The reuse and split distributions are

$$P^{v}_{\text{reuse}}(\boldsymbol{y}) = \sum_z r_v(z) \prod_{j=1}^{R} i_z(y_j),$$

$$P^{v,v'}_{\text{split}}(\boldsymbol{y}) = \left(\sum_z r_v(z) \prod_{j=1}^{R-1} i_z(y_j) \right) \left(\sum_{z'} r_{v'}(z') i_{z'}(y_R) \right).$$

Let $\gamma_=$ be their maximum symmetric log-likelihood ratio over $v = v'$ and all $\boldsymbol{y}$, and define $\gamma_{\neq}$ analogously for $v \neq v'$. Both values are computed exactly from a, b, and R.

Theorem 5.3 (Complete Search-Pattern XDP). Let $K_Q = \sum_{s \in \mathcal{S}_Q} \kappa_s$ and $u = D_Q(\boldsymbol{q}, \boldsymbol{q}')$. Define

$$\xi_{\text{SP}}(W_0, W_1) = (K_Q - u)\gamma_= + u\gamma_{\neq}.$$

For every measurable event E,

$$\Pr[\mathsf{V}_{\text{SP}}(W_0) \in E] \leq e^{\xi_{\text{SP}}(W_0, W_1)} \Pr[\mathsf{V}_{\text{SP}}(W_1) \in E],$$

with the reverse inequality obtained by exchanging W_0 and W_1. Therefore, the complete transcript satisfies $(\xi_{\text{SP}}, 0)$-XDP over the declared workload adjacency. A uniform ordinary-DP bound is $\big(K_Q \max\{\gamma_=, \gamma_{\neq}\}, 0\big)$.

Skeched Proof. Each of the u differing coordinates contributes at most $\gamma_{\neq}$. On the remaining $K_Q - u$ coordinates, the clean bits agree, but replacing a shared permanent state with an independent state changes cross-submission correlation by at most $\gamma_=$. Independence across coordinates and composition give $\xi_{\text{SP}}(W_0, W_1)$. Search trajectories, returned results, and cross-shard correspondences are post-processing of the report sequences and fixed index and incur no additional privacy cost. □

The generally nonzero $(K_Q - u)\gamma_=$ term is necessary because search-pattern privacy protects the query-reuse relation, not only the query value. Thus, ξ_{SP} is an XDP parameter over the declared workload adjacency but cannot be expressed solely using D_Q. Since $\gamma_=$ and $\gamma_{\neq}$ are calculated directly from PRR/IRR, no separate search-pattern experiment is required. The theorem assumes that no stable query identifier or memoization key is exposed.

5.2.5 Discussion. The stored-index theorem quantifies stored-data privacy. The access-pattern theorem bounds leakage from visible HNSW traversals, while the search-pattern theorem quantifies repeated-query leakage. The stored-index theorem conservatively assumes perfect cross-shard association. Appendix A.2 evaluates whether geometry-, topology-, and graph-signal-based methods can recover such correspondences in practice.

6 Evaluation

In this section, we evaluate the performance of MESS. We structure the results around two research questions:

(1) How does MESS compare with the baselines in search quality, index construction cost, latency, and communication cost?
(2) How does MESS scale to support a hundred-million-vector index?

Baselines. We compare MESS against four baselines.

- **Plaintext-HNSW.** This baseline performs non-private semantic search. It builds an HNSW index over the original embeddings using `hnswlib` and answers queries directly in plaintext. We use this to evaluate the privacy overhead of MESS.
- **Compass.** This baseline combines HNSW search with ORAM [51]. It hides the logical graph nodes accessed during adaptive search, but requires multiple encrypted block transfers and client–server interactions. We use the Compass configurations corresponding to the evaluated datasets and apply the same network settings to its reported communication cost.
- **HE-Cluster.** This baseline adopts homomorphic encryption for semantic search. It is a variant of Tiptoe [18] and is also used as a baseline in Compass [51]. It partitions N items into approximately $\sqrt{N}$ padded semantic clusters, computes encrypted similarities within the selected cluster, and returns encrypted scores for client-side ranking.
- **No-Noise.** This baseline is the same as MESS but without storage- and query-side LSHRR. We use this for ablation: the gap between Plaintext-HNSW and No-Noise captures the effect of binary multi-graph retrieval, while the gap between No-Noise and MESS captures the impact of randomized perturbation.

Datasets and metrics. We evaluate SIFT, LAION, TripClick, and MS MARCO, covering dense-vector, multimodal, and text-retrieval workloads. For SIFT and LAION, we use Recall@K, which measures the overlap between the returned and ground-truth Top-K results. For TripClick and MS MARCO, we use MRR@10, which emphasizes highly ranked relevant results. Communication is measured by total request and response bytes and the number of messages. Dataset-specific parameters, query counts, and candidate budgets are reported in Appendix A.3.

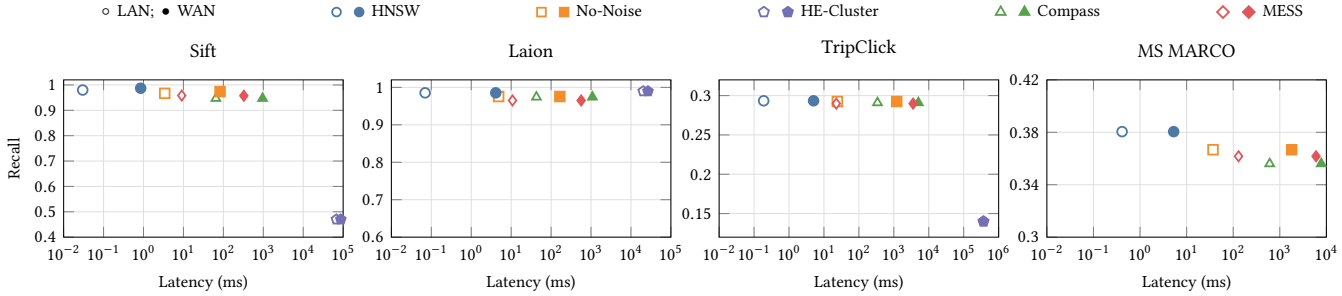
Experiment settings. All experiments run on a dual-socket machine with Hygon C86 7375 32-core processors at 2.0 GHz and 512 GB of memory. We use Linux traffic control to emulate a LAN with 3 Gbps bandwidth and 1 ms RTT and a WAN with 400 Mbps bandwidth and 80 ms RTT.

6.1 Performance Against Baselines

Fig. 4 compares the search quality of MESS and the baselines. Tab. 2 breaks down the latency and communication cost. For graph-based methods, we report results on configurations selected for high accuracy. The results for HE-Cluster are based on the configurations used in Compass [51].

Search quality. Fig. 4 shows that MESS remains close to Plaintext-HNSW: the Recall gap is below 0.05, and the MRR gap is below 0.03. We can explain this gap using No-Noise's result. Specifically, although No-Noise removes randomized perturbation, it still searches finite-length binary embeddings in Hamming space, distributes

Fig. 4: Search quality versus latency. Hollow and filled markers indicate LAN and WAN; TripClick and MS MARCO report MRR@10. HE-Cluster is included only where reported by Compass and is not always quality-matched.



Tab. 2: Latency and communication cost. HNSW is non-private; No-Noise disables perturbation; HE-Cluster and Compass use HE and ORAM, respectively; Ours denotes MESS. Two messages form one request–response round.

Dataset	Scheme	Offline Setup Phase	Online Search Latency (ms/query)					Messages per Query	Comm. Overhead (KB/query)
		Build Time (s)	Mode	Server Eval.	Trans. Lat.	Client Eval.	Total Time		
SIFT (10K, 128*d*)	HNSW [34]	44.42	WAN	0.02	0.83	–	0.85	2	0.64
			LAN	0.02	0.01	–	0.03		
	No-Noise	435.40	WAN	1.08	80.73	0.97	82.78	2	138.07
			LAN	0.99	1.52	0.89	3.40		
	HE-Cluster	–	WAN	55621.65	24083.69	7716.07	87421.42	2	744096.35
			LAN	55980.91	4250.11	7708.48	67939.52		
	Compass [51]	≥ 44.42	WAN	10.03	936.62	11.70	958.35	16.1048	11451.86
			LAN	13.74	36.85	14.43	65.02		
	Ours	512.74	WAN	1.45	323.32	1.34	**326.11**	2	788.82
			LAN	1.16	6.75	1.18	**9.09**		
LAION (1M, 512*d*)	HNSW [34]	6.53	WAN	0.02	4.11	–	4.13	2	2.14
			LAN	0.02	0.07	–	0.09		
	No-Noise	25.64	WAN	1.20	163.11	1.19	165.50	2	391.82
			LAN	1.29	2.54	1.02	4.85		
	HE-Cluster	–	WAN	16408.31	8040.37	2217.09	26665.74	2	208941.03
			LAN	16408.31	1601.69	2228.37	20238.37		
	Compass [51]	≥ 6.53	WAN	2.74	1068.67	11.18	1082.59	16.002	45989.36
			LAN	3.12	28.35	11.45	42.92		
	Ours	44.33	WAN	1.12	563.55	1.14	**565.81**	2	1303.57
			LAN	0.92	9.20	0.70	**10.82**		
TripClick (1.5M, 768*d*)	HNSW [34]	1211.97	WAN	0.13	4.92	–	5.05	2	3.14
			LAN	0.07	0.12	–	0.19		
	No-Noise	1137.13	WAN	2.88	1206.90	1.32	1211.10	2	2914.82
			LAN	2.33	20.82	1.10	24.25		
	HE-Cluster	–	WAN	325800.00	29180.00	8620.00	363590.00	2	782504.63
			LAN	358757.20	5502.80	8897.02	373157.00		
	Compass [51]	≥ 1211.97	WAN	134.82	4774.15	132.70	5041.67	18.0613	133042.91
			LAN	25.45	200.60	114.65	340.70		
	Ours	1586.19	WAN	6.46	3563.14	1.59	**3571.19**	2	8936.57
			LAN	2.97	18.01	1.62	**22.60**		
MS MARCO (8.8M, 768*d*)	HNSW [34]	10728.84	WAN	0.22	5.05	–	5.27	2	3.15
			LAN	0.18	0.23	–	0.41		
	No-Noise	8676.12	WAN	4.33	1795.45	1.53	1801.31	2	4468.82
			LAN	3.03	32.33	1.52	36.88		
	HE-Cluster	–	WAN	–	–	–	–	–	–
			LAN	–	–	–	–		
	Compass [51]	≥ 10728.84	WAN	122.32	7445.57	195.61	7763.50	18.0209	226657.48
			LAN	64.01	361.85	181.90	607.76		
	Ours	11741.53	WAN	4.20	5997.02	1.45	**6002.67**	2	18066.32
			LAN	6.98	120.47	2.84	**130.29**		

Tab. 3: SIFT100M performance under different query-side perturbation parameters.

Dataset	Build Time	p	Mode	Online Search Performance				Candidate	Communication
				Server Eval.	Trans.Lat.	Client Dec.&Rank	Total Time		
SIFT100M	26.19 h	0.00	WAN	3.21	888.89	1.63	893.73	4000	2123.54
			LAN	1.98	21.52	1.36	24.86	4000	2123.54
		0.08	WAN	8.61	2031.79	2.45	2042.85	9500	5069.04
			LAN	3.43	46.91	2.19	52.53	9500	5069.04
		0.10	WAN	14.33	2666.87	2.86	2684.06	12500	6678.79
			LAN	11.40	57.53	2.55	71.48	12500	6678.79
		0.12	WAN	41.78	4225.49	3.90	4271.17	20000	10686.04
			LAN	10.99	100.49	5.45	116.93	20000	10686.04

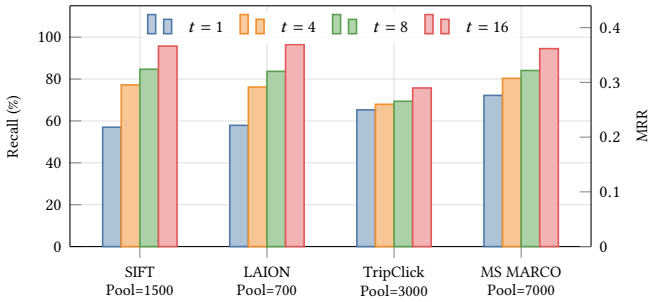


Fig. 5: Effect of the number of selected shards t on search quality with $M = 64$ and a fixed candidate pool for each dataset. SIFT and LAION are evaluated using Recall, while TripClick and MS MARCO are evaluated using MRR.

items over several graph shards, aggregates candidates, transfers encrypted payloads, removes duplicates, and reranks on the client. Its gap to Plaintext-HNSW represents the effect of searching binary multi-graph without privacy noise. Its gap to MESS is attributable to LSHRR and the larger candidate set needed to compensate for randomized bit flips. We can see that adding LSHRR leads to a quality change of at most 0.011. This small quality change comes at the cost of efficiency, because MESS must retrieve more candidates to maintain quality, increasing communication cost.

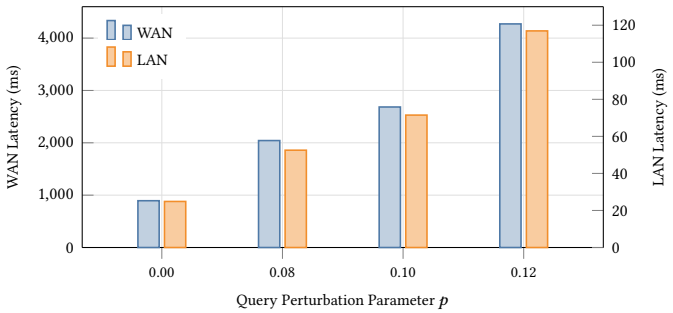
Comparison with cryptographic baselines. Compass achieves accuracy close to that of non-private search, but ORAM-protected adaptive traversal requires repeated accesses to encrypted blocks. Tab. 2 shows that MESS is 4.0–15.1× faster under LAN and 1.3–2.9× faster under WAN. The largest LAN improvement appears on TripClick, where the cost of interactive traversal is especially high. The WAN advantage is smaller because transfer time dominates in both systems as network speed decreases. HE-Cluster shows a different bottleneck. On the three datasets for which Compass reports results, encrypted similarity evaluation and encrypted score transfer require approximately 20–373 seconds and 209–783 MB per query. Although its quality is competitive on LAION, the results indicate significant computational and communication overhead that is orders of magnitude higher than MESS. In summary, the two baselines that adopt cryptographic approaches suffer from high overhead: HE-Cluster incurs significant overhead in vector computation and produces large ciphertexts, and Compass incurs large communication overhead.

Impact of t. Fig. 5 evaluates the impact of t on search quality, when M is fixed at 64. It can be seen that search quality improves as more shards are selected. We also note that larger t also increases the number of shard-local entries and the associated storage cost. Increasing t from 1 to 16 raises Recall from 57.0% to 95.73% on SIFT and from 57.9% to 96.5% on LAION. TripClick MRR increases from 0.250 to 0.290, while MS MARCO MRR increases from 0.276 to 0.362. This is because a larger t places each vector in more independently perturbed graph views, increasing the probability that it retains a favorable rank in at least one shard. The gains are more pronounced for the Recall metric than for the MRR metric, because the latter considers the vector's final position after exact reranking.

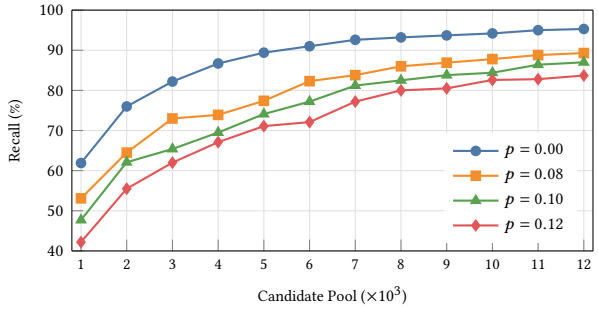
Latency breakdown. For MESS under LAN, server search remains below 7 ms and client decryption and reranking below 3 ms across the four datasets. Therefore, both Hamming graph traversal and local cryptographic processing are efficient. The remaining cost is encrypted candidate transfer, which becomes significant under WAN: on MS MARCO, this transfer accounts for 5997.02 ms of the 6002.67 ms end-to-end latency. Compass is also network-bound under WAN, but incurs high computation cost due to ORAM operations. These results suggest that optimizing candidate set and payload size are interesting future extensions for MESS.

Offline cost. The offline cost of MESS includes shard-specific hashing, independent perturbation, payload encryption, and construction of multiple HNSW shards. It is therefore notably higher than constructing one plaintext, as shown in Tab. 2. However, this cost is incurred once and can be amortized over subsequent queries.

Insertion and deletion. MESS supports incremental updates without rebuilding the complete index. For insertion, the client computes and perturbs the new item's shard-specific binary embeddings, encrypts its payload, and sends it to the $t = 16$ shards. The server then inserts the perturbed item into the shards. Deletion is performed lazily: the client identifies the shard-local labels, and the server marks the corresponding nodes as inactive, thereby excluding them from future searches. On MS MARCO, insertion takes 96.16 ms over LAN and 351.29 ms over WAN, while deletion takes 1.33 ms and 80.35 ms, respectively. Their communication costs are 3.09 KB and 0.09 KB.



(a) Total search latency.



(b) Recall under different candidate-pool sizes.

Fig. 6: SIFT100M performance under different query-side perturbation parameters: (a) total search latency under LAN and WAN settings; and (b) Recall@10 under different candidate-pool sizes.

6.2 Scalability

We evaluate the performance of MESS at scale, using a large dataset (SIFT100M) created by selecting the first 100 million vectors from SIFT1B. We note that neither Compass [51] nor iptoe [18] report results at this scale. We focus only on the online-phase performance and note that the offline phase takes 26.19 hours.

Fig. 6 and Tab. 3 show the effect of the query-side perturbation probability p on Recall@10, candidate volume, latency, and communication. With 12,000 candidates, increasing p from 0 to 0.12 reduces Recall@10 from 95.3% to 83.7%. It can be seen that the gap decreases with larger candidate sets. However, as the candidate sets grow from 4,000 to 20,000 candidates, the communication cost increases 5.0×. These results demonstrate the accuracy and efficiency trade-off. The latency breakdown in Tab. 3 shows that the costs of graph search and client reranking remain small compared to that of encrypted candidate transfer. In particular, at $p = 0.08$, MESS processes SIFT100M in 52.53 ms/query under LAN; even at $p = 0.12$, LAN latency remains below 120 ms/query. Under WNA, the latency increases from 893.73 ms to 4271.17 ms due to large payloads. These results confirm that MESS is practical for large-scale datasets. They highlight the fundamental trade-off: stronger query perturbation increases rank uncertainty, therefore requires larger candidate sets and incurs higher communication overhead. The results indicate that reducing encrypted payload size or improving shard-level candidate filtering is more beneficial than optimizing the relatively small graph-search and reranking costs.

7 Related Work

Vector Databases and ANN Search. Vector databases use approximate nearest-neighbor (ANN) indexes to support large-scale, high-dimensional embedding search. HNSW provides low-latency and high-recall graph traversal, while FAISS and SPANN study GPU acceleration and memory–disk hybrid indexing at billion-vector scale [12, 24, 34]. Recent systems also consider database-level requirements: HAKES supports high-recall search under concurrent updates, while VBase integrates vector similarity search with relational query processing [20, 49]. These systems target performance and scalability of plaintext vector search, whereas MESS targets privacy. While indexes other than HNSW can be used in MESS, our current security analysis assumes the use of graph indexes. Extending MESS to support other indexes is left as future work.

Encrypted Semantic Retrieval. Existing works reduce secure candidate-generation costs through clustering, LSH, tree indexes, or stronger deployment assumptions. Tiptoe and Wally use clustering, with Wally additionally combining epochs and differential privacy [7, 18]. Preco uses LSH with two non-colluding servers, while VCC24 combines tree indexing and k-means under information-theoretic security [39, 44]. SANNS uses DORAM and garbled circuits for secure Top-K selection [11], whereas trusted-hardware approaches introduce additional trust assumptions [6, 45]. Other works adopt Private Information Retrieval (PIR). In particular, Preco notes that replacing its non-colluding servers with single-server PIR would increase overhead, while Panther combines PIR2A with secret sharing for private access and secure Top-K selection [29, 39].

Cryptographic solutions use order- or property-preserving encryption for distance comparison, but the ciphertext may leak metric relations [21, 32, 48]. Compass uses ORAM to hide HNSW accesses [51], while PACMANN combines PIR with a secure neighborhood graph [50]. These solutions provide stronger obliviousness but suffer significant overhead from cryptographic operations. In contrast, MESS searches server-visible HNSW shards over perturbed binary embeddings, with well-defined stored-data, access-pattern, and search-pattern leakage.

8 Conclusion

We presented MESS that enables semantic search with privacy, accuracy, and efficiency. The system adopts a differential privacy approach, allowing search to be performed over perturbed vectors while bounding leakage of stored data, access patterns, and search patterns. MESS proposes a multi-graph design that compensates for the accuracy loss from perturbation. The evaluation results show that it outperforms cryptographic baselines, reducing communication costs by up to 35.28× compared to Compass. The system remains practical at scale, achieving 52.53 ms/query on a dataset of 100M vectors.

References

[1] [n. d.]. AI Database that developers love. https://weaviate.io/. Accessed: 2026.
[2] [n. d.]. Amazon OpenSearch Service. https://aws.amazon.com/opensearch-service/serverless-vector-database/. Accessed: 2026.
[3] [n. d.]. Vector Search. https://docs.cloud.google.com/gemini-enterprise-agent-platform/build/vector-search/overview. Accessed: 2026.
[4] Ishtiyaque Ahmad, Laboni Sarker, Divyakant Agrawal, Amr El Abbadi, and Trinabh Gupta. 2021. Coeus: A system for oblivious document ranking and

retrieval. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 672–690.

[5] Kinan Dak Albab, Rawane Issa, Mayank Varia, and Kalman Graffi. 2022. Batched differentially private information retrieval. In *31st USENIX Security Symposium (USENIX Security 22)*. 3327–3344.

[6] Ghous Amjad, Seny Kamara, and Tarik Moataz. 2019. Forward and backward private searchable encryption with SGX. In *Proceedings of the 12th European Workshop on Systems Security*. 1–6.

[7] Hilal Asi, Fabian Boemer, Nicholas Genise, Muhammad Haris Mughees, Tabitha Ogilvie, Rehan Rishi, Kunal Talwar, Karl Tarbe, Akshay Wadia, Ruiyu Zhu, et al. 2024. Scalable private search with wally. *arXiv preprint arXiv:2406.06761* (2024).

[8] Martin Aumüller, Anders Bourgeat, and Jana Schmurr. 2020. Differentially Private Sketches for Jaccard Similarity Estimation. In *Similarity Search and Applications - 13th International Conference, SISAP 2020*. Springer, 18–32. doi:10.1007/978-3-030-60936-8_2

[9] Laura Blackstone, Seny Kamara, and Tarik Moataz. 2019. Revisiting leakage abuse attacks. *Cryptology ePrint Archive* (2019).

[10] Guoxing Chen, Ten-Hwang Lai, Michael K Reiter, and Yinqian Zhang. 2018. Differentially private access patterns for searchable symmetric encryption. In *IEEE INFOCOM 2018-IEEE conference on computer communications*. IEEE, 810–818.

[11] Hao Chen, Ilaria Chillotti, Yihe Dong, Oxana Poburinnaya, Ilya Razenshteyn, and M Sadegh Riazi. 2020. {SANNS}: Scaling up secure approximate {k-Nearest} neighbors search. In *29th USENIX Security Symposium (USENIX Security 20)*. 2111–2128.

[12] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. In *Advances in Neural Information Processing Systems*, Vol. 34. 5199–5212.

[13] Thomas Cover and Peter Hart. 1967. Nearest neighbor pattern classification. *IEEE transactions on information theory* 13, 1 (1967), 21–27.

[14] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. 2006. Searchable symmetric encryption: improved definitions and efficient constructions. In *Proceedings of the 13th ACM conference on Computer and communications security*. 79–88.

[15] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. 2014. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 1054–1067.

[16] Natasha Fernandes, Yusuke Kawamoto, and Takao Murakami. 2021. Locality Sensitive Hashing with Extended Differential Privacy. In *Computer Security – ESORICS 2021, Part II (Lecture Notes in Computer Science, Vol. 12973)*. Springer, 563–583. doi:10.1007/978-3-030-88428-4_28

[17] Donniell E. Fishkind, Sancar Adali, Heather G. Patsolic, Lingyao Meng, Digvijay Singh, Vince Lyzinski, and Carey E. Priebe. 2019. Seeded Graph Matching. *Pattern Recognition* 87 (2019), 203–215. doi:10.1016/j.patcog.2018.09.014

[18] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. 2023. Private web search with tiptoe. In *Proceedings of the 29th symposium on operating systems principles*. 396–416.

[19] Alexandra Henzinger, Matthew M. Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. 2023. One server for the price of two: simple and fast single-server private information retrieval. In *Proceedings of the 32nd USENIX Conference on Security Symposium*. USENIX Association, USA.

[20] Guoyu Hu, Shaofeng Cai, Tien Tuan Anh Dinh, Zhongle Xie, Cong Yue, Gang Chen, and Beng Chin Ooi. 2025. HAKES: Scalable Vector Database for Embedding Search Service. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3049–3062. doi:10.14778/3746405.3746427

[21] Zhengbai Huang, Meng Zhang, and Yi Zhang. 2019. Toward Efficient Encrypted Image Retrieval in Cloud Environment. *IEEE Access* 7 (2019), 174541–174550. doi:10.1109/ACCESS.2019.2957497

[22] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of the thirtieth annual ACM symposium on Theory of computing*. 604–613.

[23] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118* (2021).

[24] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. doi:10.1109/TBDATA.2019.2921572

[25] Darya Kaviani, Alp Eren Ozdarendeli, Jinhao Zhu, Yu Ding, and Raluca Ada Popa. 2026. Opal: Private Memory for Personal AI. *arXiv preprint arXiv:2604.02522* (2026).

[26] Ehsan Kazemi, Seyed Hamed Hassani, and Matthias Grossglauser. 2015. Growing a Graph Matching from a Handful of Seeds. *Proceedings of the VLDB Endowment* 8, 10 (2015), 1010–1021. doi:10.14778/2794367.2794371

[27] Weihao Kong and Wu-Jun Li. 2012. Isotropic hashing. *Advances in neural information processing systems* 25 (2012).

[28] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2026. SemBench: A Benchmark for Semantic Query Processing Engines. *Proc. VLDB Endow.* 19, 8 (July 2026), 1754–1767. doi:10.14778/3811243.3811249

[29] Jingyu Li, Zhicong Huang, Min Zhang, Cheng Hong, Jian Liu, Tao Wei, and Wenguang Chen. 2025. Panther: Private approximate nearest neighbor search in the single server setting. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 365–379.

[30] Hang Liu, Anna Scaglione, and Hoi-To Wai. 2024. Blind Graph Matching Using Graph Signals. *IEEE Transactions on Signal Processing* 72 (2024), 1766–1781. doi:10.1109/TSP.2024.3382840

[31] Yingfan Liu, Yandi Zhang, Jiadong Xie, Hui Li, Jeffrey Xu Yu, and Jiangtao Cui. 2025. Privacy-Preserving Approximate Nearest Neighbor Search on High-Dimensional Data. In *Proceedings of the IEEE 41st International Conference on Data Engineering (ICDE 2025)*. 3017–3029. doi:10.1109/ICDE65448.2025.00226

[32] Yingfan Liu, Yandi Zhang, Jiadong Xie, Hui Li, Jeffrey Xu Yu, and Jiangtao Cui. 2025. Privacy-preserving approximate nearest neighbor search on high-dimensional data. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 3017–3029.

[33] David G Lowe. 2004. Distinctive image features from scale-invariant keypoints. *International journal of computer vision* 60, 2 (2004), 91–110.

[34] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[35] Arvind Narayanan and Vitaly Shmatikov. 2009. De-anonymizing Social Networks. In *2009 30th IEEE Symposium on Security and Privacy*. IEEE, 173–187. doi:10.1109/SP.2009.22

[36] Simon Oya and Florian Kerschbaum. 2021. Hiding the access pattern is not enough: Exploiting search pattern leakage in searchable encryption. In *30th USENIX security symposium (USENIX Security 21)*. 127–142.

[37] M. Sadegh Riazi, Beidi Chen, Anshumali Shrivastava, Dan Wallach, and Farinaz Koushanfar. 2019. Sub-Linear Privacy-Preserving Near-Neighbor Search. Cryptology ePrint Archive, Paper 2019/1222. https://eprint.iacr.org/2019/1222

[38] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*. 285–295.

[39] Sacha Servan-Schreiber, Simon Langowski, and Srinivas Devadas. 2022. Private approximate nearest neighbor search with sublinear communication. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 911–929.

[40] Zhiwei Shang, Simon Oya, Andreas Peter, and Florian Kerschbaum. 2021. Obfuscated Access and Search Patterns in Searchable Encryption. In *Proceedings of the 28th Annual Network and Distributed System Security Symposium (NDSS 2021)*.

[41] Sivic and Zisserman. 2003. Video Google: A text retrieval approach to object matching in videos. In *Proceedings ninth IEEE international conference on computer vision*. IEEE, 1470–1477.

[42] Paul Swoboda, Dagmar Kainmüller, Ashkan Mokarian, Christian Theobalt, and Florian Bernard. 2019. A Convex Relaxation for Multi-Graph Matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11156–11165. doi:10.1109/CVPR.2019.01141

[43] Warren S Torgerson. 1952. Multidimensional scaling: I. Theory and method. *Psychometrika* 17, 4 (1952), 401–419.

[44] Sajani Vithana, Martina Cardone, and Flavio P Calmon. 2024. Private approximate nearest neighbor search for vector database querying. In *2024 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 3666–3671.

[45] Viet Vo, Shangqi Lai, Xingliang Yuan, Surya Nepal, and Joseph K Liu. 2021. Towards efficient and strong backward private searchable encryption with secure enclaves. In *International Conference on Applied Cryptography and Network Security*. Springer, 50–75.

[46] Yichuan Wang, Zhifei Li, Shu Liu, Yongji Wu, Ziming Mao, Yilong Zhao, Xiao Yan, Zhiying Xu, Yang Zhou, Ion Stoica, et al. 2025. LEANN: A Low-Storage Vector Index. *arXiv preprint arXiv:2506.08276* (2025).

[47] Kilian Q Weinberger and Lawrence K Saul. 2009. Distance metric learning for large margin nearest neighbor classification. *Journal of machine learning research* 10, 2 (2009).

[48] Wai Kit Wong, David Wai-lok Cheung, Ben Kao, and Nikos Mamoulis. 2009. Secure kNN computation on encrypted databases. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of data*. 139–152.

[49] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*. 377–395.

[50] Mingxun Zhou, Elaine Shi, and Giulia Fanti. 2024. Pacmann: Efficient private approximate nearest neighbor search. In *The Thirteenth International Conference on Learning Representations*.

[51] Jinhao Zhu, Liana Patel, Matei Zaharia, and Raluca Ada Popa. 2025. Compass: Encrypted semantic search with high accuracy. In *19th USENIX Symposium on*

A Detailed Accounting and Empirical Leakage Evaluation

This appendix complements Section 5.2. The XDP definitions, mechanism-level guarantees, and proofs are given in the main text. Here, we describe the numerical privacy accountants and evaluate how much multi-shard XDP evidence can be organized by concrete cross-shard inference procedures.

A.1 Implementation of the Privacy Accountants

A.1.1 Stored-Data Accountant. For a real challenge pair $q = (\boldsymbol{x}_0, \boldsymbol{x}_1)$, we compute the fixed-ISO-LSH distance $d_s(q) = d_H(H_s(\boldsymbol{x}_0), H_s(\boldsymbol{x}_1))$ for every frozen shard mapping. Under uniform t-of-M assignment, let $w_q(u)$ be the probability that the aggregate distance over the selected shards equals u.

We compute $w_q(u)$ by dynamic programming rather than enumerating all $\binom{M}{t}$ shard sets. Let $F_i(j,u)$ count the subsets of size j among the first i shards whose aggregate distance is u. Starting from $F_0(0,0) = 1$, we apply $F_i(j,u) = F_{i-1}(j,u) + F_{i-1}(j-1, u-d_i(q))$ and obtain $w_q(u) = F_M(t,u)/\binom{M}{t}$. For identifier-fixed assignment, w_q is a point mass at the realized aggregate distance.

For every u, we evaluate the exact randomized-response privacy-loss distribution in log space and obtain its hockey-stick profile $\delta_u^{\mathrm{RR}}(\varepsilon)$. The route-averaged profile is then formed using $w_q(u)$ and inverted by bisection to obtain the pair-specific XDP value at target δ_0. Both likelihood-ratio directions are evaluated.

A.1.2 Query Accountants. For access-pattern privacy, the value in Theorem 5.2 is computed directly from the PRR probability a, IRR probability b, repetition count R, and fixed-hash query distance D_Q. Log-sum-exp arithmetic is used when evaluating η_R, jointly accounting for the memoized PRR state and all fresh IRR reports.

For search-pattern privacy, the reuse and split distributions depend only on the number of ones among the first $R-1$ reports and the final report bit. We enumerate these possibilities to compute $\gamma_=$ and $\gamma_\neq$ exactly. No additional access- or search-trace experiment is required because, under the stated system model, the HNSW trace is post-processing of the perturbed reports and fixed index. This conclusion excludes stable query identifiers, routing based on clean queries, and external side channels.

A.2 Empirical Cross-Shard Inference Evaluation

A.2.1 Evaluation Objective. The formal complete-view theorem grants the server perfect association of all shard-local entries belonging to the target item. This assumption is conservative because the implementation exposes no direct cross-shard correspondence: different shards use different ISO-LSH mappings and local handles, randomized response is sampled independently, and payload ciphertexts use fresh encryption randomness.

The experiment asks a more practical question: starting from one known target entry, how many additional target entries can a specified inference procedure identify, and how much XDP evidence can it organize after combining the recovered observations? The experiment does not replace the mechanism-level XDP theorem. It measures only the leakage achieved by the evaluated procedures, challenge pairs, auxiliary seeds, and system configuration.

A.2.2 Neighboring Worlds. For each challenge pair $q = (\boldsymbol{x}_0, \boldsymbol{x}_1)$, we construct two neighboring databases with identical background records and identifiers. They differ only in the embedding of one target identifier $i^\star$: world 0 contains $(i^\star, \boldsymbol{x}_0)$ and world 1 contains $(i^\star, \boldsymbol{x}_1)$. Every trial reruns t-of-M assignment, storage-side randomized response, shard-local graph construction, and cross-shard inference. The assignment mechanism is identical in the two worlds and independent of the protected embedding. Calibration and held-out trials use disjoint randomness. Ground-truth cross-shard correspondences and world labels are never supplied to the inference procedures.

A.2.3 Cross-Shard Inference Procedures. We evaluate four complementary procedures. MDS constructs shard-local Hamming-distance matrices and obtains low-dimensional coordinates using classical multidimensional scaling [43]; the coordinates are then aligned using fixed background seeds. Topology compares node degree, neighbor-degree statistics, and multi-hop HNSW structure, drawing on topology-based de-anonymization and seeded graph matching [17, 26, 35]. GSGM propagates perturbed-code signals over each graph and compares the resulting graph-filtered descriptors [30]. Joint standardizes and combines the three scores and applies cycle-consistency checks across shard triples [42].

The evaluator provides one known target entry in a reference shard and a fixed fraction of background correspondences as auxiliary seeds. The remaining target entries are hidden. For each destination shard, a procedure ranks all non-seed candidates using only server-visible perturbed codes and graph structure. It then retains the highest-confidence candidates until the public selected-shard count t is reached. Incorrect candidates are not removed using ground truth, and no learned classifier is used. All descriptors, normalizations, weights, candidate-count grids, and stopping rules are fixed before held-out evaluation.

A.2.4 From Cross-Shard Association to XDP. The central step is to convert the inferred cross-shard associations into a fixed-length multi-shard observation. For each trial, the t selected κ-bit blocks are concatenated into a reconstructed vector of length $t\kappa$.

If an inferred entry is the correct target entry in shard s, its block contains genuine randomized-response evidence about the challenge pair and contributes the actual fixed-hash distance $d_s(q)$. If the inferred entry is incorrect, the selected background block remains in the reconstruction. Such a block is expected to behave like an unrelated random code, whose normalized Hamming distance is approximately $1/2$. It therefore contributes no direct target-dependent RR distance under the common-null model.

Ground truth is used only after inference to define $C_{\mathcal{A},s} = 1$ when method $\mathcal{A}$ correctly identifies the target in shard s, and zero otherwise. The direct target distance organized by the method is $U_{\mathcal{A},q} = \sum_s C_{\mathcal{A},s} d_s(q)$. Thus, a successful association contributes its actual shard-specific distance, while a false or missing association contributes no target distance. False blocks are nevertheless retained when checking whether their empirical normalized distance is close to the random-code reference $1/2$.

Across held-out trials, let $\widehat{w}_{\mathcal{A},q}(u)$ be the empirical distribution of $U_{\mathcal{A},q}$. We combine this distribution with the exact RR profile:

$$\delta^{\text{link}}_{\mathcal{A},q}(\varepsilon) = \sum_u \widehat{w}_{\mathcal{A},q}(u)\delta^{\text{RR}}_u(\varepsilon),$$

$$\xi^{\text{link}}_{\mathcal{A},q}(\delta_0) = \inf\left\{\varepsilon : \delta^{\text{link}}_{\mathcal{A},q}(\varepsilon) \le \delta_0\right\}.$$

This construction directly reflects the experimental objective. If only the supplied reference entry is retained, the result contains approximately single-shard XDP evidence. Each additional correct association adds its actual fixed-hash distance and increases the reconstructed multi-shard XDP. If all target entries are identified, the reconstructed view approaches the idealized complete-view observation. Importantly, the association success rate is not treated as a subsampling probability and is not multiplied directly by a privacy budget. Instead, the experiment first constructs the empirical distribution of the total recovered distance and then applies the exact RR privacy-loss accountant. Consequently, $\xi^{\text{link}}_{\mathcal{A},q}$ remains a distance-aware XDP quantity.

Because the candidate-selection procedure is data dependent, ξ^{link} should be interpreted as the direct RR evidence organized by the specified inference method, rather than a mechanism-level guarantee against arbitrary algorithms.

A.2.5 Configuration. The experiments use $M = 64$ shards, $t = 16$ selected shards per item, $\kappa = 128$ bits per shard, and storage-side flip probability $p_D = 0.08$. At angular radius $d_\theta \le 0.05$, each valid configuration contains up to five real challenge pairs, one known target entry, and 20% auxiliary background seeds. Every challenge pair uses 30 calibration trials and 300 held-out trials per world.

We evaluate index prefixes containing 5,000, 10,000, and 20,000 vectors. MS MARCO and SIFT contain five eligible pairs in each valid configuration, whereas TripClick at 10k contains two. The prior random-LSH regression value $\xi = 20.041$ for $p = 0.21$, $\kappa = 128$, $d_\theta = 0.05$, and $\delta_0 = 0.01$ is used only as an implementation regression test. It is not substituted for the fixed-ISO-LSH accountant used in the evaluation.

A.2.6 Results and Analysis. At $d_\theta \le 0.05$ and $\delta_0 = 0.01$, the largest reconstructed XDP value over MDS, Topology, GSGM, and Joint ranges from 31.586 to 78.110 across the valid configurations. These values are above the corresponding single-shard diagnostic range of 24.423–36.635, showing that the evaluated procedures can sometimes organize evidence from more than the supplied reference shard. However, they remain far below the idealized complete-16-shard privacy-loss thresholds of 200.272–410.314. The practical inference procedures therefore recover only part of the information granted to the perfect-association adversary in the formal theorem.

For MS MARCO, the largest reconstructed value is 71.812 at 5k using MDS, 60.444 at 10k using MDS, and 70.588 at 20k using GSGM. These results show that residual geometry is effective at smaller scales, while graph-signal information becomes competitive at 20k. The decrease from 5k to 10k and the subsequent increase at 20k also show that reconstructed leakage is not determined by database size alone. For SIFT, the maximum increases from 60.921 at 5k to 69.692 at 10k and 78.110 at 20k. GSGM produces the largest values at 5k and 10k, whereas Joint is strongest at 20k. SIFT therefore exhibits the clearest increase in organized cross-shard evidence as the evaluated prefix grows, although this observation is specific to the sampled challenge pairs and graph instances. TripClick produces a maximum of 31.586 at 10k using Joint, close to the single-shard range and substantially below the MS MARCO and SIFT results. Because only two qualifying challenge pairs are available, this value should not be interpreted as a general property of the complete TripClick dataset.

No inference procedure dominates all datasets and scales. MDS is strongest in two valid configurations, GSGM in three, and Joint in two. Taking the maximum over the four procedures is therefore necessary, since reporting Joint alone would underestimate the leakage achieved in several configurations. The normalized distances of incorrectly selected blocks are generally close to 1/2, supporting the common-null assumption that failed associations behave approximately like unrelated binary codes. The reconstructed XDP is also non-monotone in database size because the eligible challenge pairs, background candidate pools, selected shards, and independently constructed graphs change across configurations.

LAION does not contribute a valid value because its 5k, 10k, and 20k runs terminated on an invalid angular-distance value. TripClick at 5k contains no eligible real pair, while its 20k run terminated on the same validation error. These configurations are excluded rather than assigned extrapolated privacy values.

Overall, the experiment supports two conclusions. First, cross-shard inference can increase the XDP evidence available beyond a single known shard. Second, the evaluated procedures remain substantially weaker than the perfect association assumed by the complete-view theorem. The reported ξ^{link} values therefore quantify the practical leakage achieved by the specified inference procedures, while the mechanism-level theorem remains the formal security statement.

A.3 Experimental Parameters

For a fair ablation comparison, the No-Noise baseline and MESS use the same multi-graph construction and HNSW parameters. Unless otherwise specified, we set the number of graph shards to $M = 64$, and route each database point to $t = 16$ selected shards. The HNSW construction parameter is fixed to `efConstruction` = 300 across all four datasets. Compass is evaluated using the configuration reported in the original Compass paper.

For candidate generation, No-Noise uses smaller candidate pools because it searches over unperturbed binary embeddings. Its candidate pool ranges are 250–400 on SIFT, 200–300 on LAION, 900–1050 on TripClick, and 1400–1600 on MS MARCO. In contrast, MESS uses larger candidate pools to compensate for query-side and storage-side perturbation: 1400–1600 on SIFT, 700–800 on LAION, 2900–3200 on TripClick, and 6000–6500 on MS MARCO.

B Correctness of the MESS

B.1 Distribution of the Shard-Level Rank

In this appendix, we derive the mean, variance, and asymptotic normal approximation of the shard-level rank used in the correctness analysis. We also explain its connection to the empirical Recall and MRR models.

Proposition 3 (Poisson Binomial Model of Shard-Level Rank). Consider one queried shard s. Let $Rank_s$ denote the absolute

Public Parameters: dataset size N, vector dimension d, number of graph shards M, routing multiplicity t, shard hash lengths $\{\kappa_m\}_{m=1}^{M}$, storage-side flip probability p_D, query-side PRR flip probability p_{PRR}, query-side IRR flip probability p_{IRR}, per-shard candidate budget P, final result size K, and HNSW parameters.
Entities: Client C and cloud server $\mathcal{S}$ in the semi-honest setting.
Input: At setup, C holds a plaintext database $\mathcal{D} = \{(\boldsymbol{x}_i, \mathrm{ID}_i)\}_{i=1}^{N}$, a fixed mapping-training set $\mathcal{D}_{\mathrm{train}}$ that is public or disjoint from $\mathcal{D}$, shard seeds $\{\sigma_m\}_{m=1}^{M}$, a routing seed σ_{route}, and shard encryption keys $\{sk_m\}_{m=1}^{M}$. At query time, C holds a query embedding $\boldsymbol{q}$ and a local memoization cache C_{memo}.
Server State: After setup, $\mathcal{S}$ stores the outsourced multi-graph index $\mathbb{G} = \{\mathcal{G}^{(m)}\}_{m=1}^{M}$ and the attached encrypted payloads.
Output: C outputs the identifiers and exact ℓ_2 distances of the Top-K reranked candidates.

[Initialization]

1: For each shard m, C invokes IsoHash on $\mathcal{D}_{\mathrm{train}}$ using shard seed σ_m to derive the shard-specific hash state $\mathrm{st}_{\mathrm{hash}}^{(m)}$. It obtains $\mathrm{st}_{\mathrm{hash}} = \{\mathrm{st}_{\mathrm{hash}}^{(m)}\}_{m=1}^{M}$.
2: For each database point $(\boldsymbol{x}_i, \mathrm{ID}_i)$, C obtains its routed shard set $\mathcal{S}_i \leftarrow \mathsf{Route}(\mathrm{ID}_i, M, t; \sigma_{\mathrm{route}})$. For each shard $m \in \mathcal{S}_i$, C computes $\boldsymbol{v}_{i,m} \leftarrow \mathsf{Hash}(\boldsymbol{x}_i; \mathrm{st}_{\mathrm{hash}}^{(m)})$, perturbs it as $\widetilde{\boldsymbol{v}}_{i,m} \leftarrow \mathsf{LSHRR}(\boldsymbol{v}_{i,m}, p_D)$, assigns a fresh unique shard-local label $\ell_{i,m}$, samples fresh encryption randomness $\rho_{i,m}$, and computes $C_{i,m} \leftarrow \mathsf{Enc}_{sk_m}(\mathrm{ID}_i \| \boldsymbol{x}_i; \rho_{i,m})$. It uploads $(\ell_{i,m}, \widetilde{\boldsymbol{v}}_{i,m}, C_{i,m})$ to $\mathcal{S}$ and stores $\{(m, \ell_{i,m})\}_{m \in \mathcal{S}_i}$ locally for deletion.
3: For each shard m, $\mathcal{S}$ constructs an HNSW graph shard $\mathcal{G}^{(m)}$ over the received perturbed binary embeddings using Hamming distance. Each node is indexed by its shard-local label and stores the corresponding encrypted payload.

[Query Generation]

4: Given a query embedding $\boldsymbol{q}$, C computes a shard-specific query binary embedding for each shard: $\boldsymbol{v}_q^{(m)} \leftarrow \mathsf{Hash}(\boldsymbol{q}; \mathrm{st}_{\mathrm{hash}}^{(m)})$ for $m \in [M]$.
5: For each shard m, C checks the local memoization cache. If a cached PRR representation exists, it retrieves $\overline{\boldsymbol{v}}_q^{(m)} \leftarrow C_{\mathrm{memo}}[\boldsymbol{q}, m]$. Otherwise, it samples $\overline{\boldsymbol{v}}_q^{(m)} \leftarrow \mathsf{LSHRR}(\boldsymbol{v}_q^{(m)}, p_{\mathrm{PRR}})$ and stores it in C_{memo}.
6: Before transmission, C applies instantaneous perturbation to each memoized representation: $\widetilde{\boldsymbol{v}}_q^{(m)} \leftarrow \mathsf{LSHRR}(\overline{\boldsymbol{v}}_q^{(m)}, p_{\mathrm{IRR}})$. It sends the perturbed query binary embeddings $\{\widetilde{\boldsymbol{v}}_q^{(m)}\}_{m=1}^{M}$ to $\mathcal{S}$ as the retrieval request.

[Server-Side Search]

7: For each shard m, $\mathcal{S}$ performs HNSW search over $\mathcal{G}^{(m)}$ using $\widetilde{\boldsymbol{v}}_q^{(m)}$ as the query binary embedding and obtains a shard-level candidate-label set $\mathrm{Cand}^{(m)}$ with budget P.
8: $\mathcal{S}$ collects the encrypted payloads associated with the returned labels and sends the aggregated response $\mathrm{Cand} = \bigcup_{m=1}^{M} \{(m, C_{i,m}) : \ell_{i,m} \in \mathrm{Cand}^{(m)}\}$ to C. Each ciphertext is accompanied by its shard index for selecting the corresponding decryption key.

[Local Decryption and Exact Reranking]

9: For each pair $(m, C_{i,m})$ in the returned response, C decrypts the payload using the corresponding shard key: $(\mathrm{ID}_i, \boldsymbol{x}_i) \leftarrow \mathsf{Dec}_{sk_m}(C_{i,m})$.
10: C removes duplicate candidates caused by multi-graph routing, computes the exact Euclidean distance between $\boldsymbol{q}$ and each recovered embedding $\boldsymbol{x}_i$, and reranks the candidates according to their exact distances.
11: Finally, C outputs the Top-K reranked candidates together with their identifiers and exact distances.

Fig. 7: The complete protocol of MESS, including initialization, PRR/IRR query generation, server-side HNSW search, and client-side decryption and reranking.

rank of the target item after storage-side and query-side perturbation. Let $N(d)$ be the number of non-target database items whose clean Hamming distance to the query is d, where $d \in \{0, \ldots, \kappa\}$. Since every item is routed uniformly to t of the M shards, the expected number of distance-d items in one shard is $\mathbb{E}[N_s(d)] = (t/M)N(d)$.

For a background item at distance d, let $I_{i,d} = \mathbf{1}\{\widetilde{D}_{i,d} < \widetilde{D}_{d_s^*}\}$ be its overtaking indicator, where d_s^* is the clean distance of the target. The effective query flip probability is $p_q = p_{\mathrm{PRR}}(1 - p_{\mathrm{IRR}}) + p_{\mathrm{IRR}}(1 - p_{\mathrm{PRR}})$, and the compound flip probability is $p_\oplus = p_D(1 - p_q) + p_q(1 - p_D)$. The Gaussian approximation gives

$$P_{\mathrm{overtake}}(d) \approx \Phi\left(\frac{(d_s^* - d)(1 - 2p_\oplus)}{\sqrt{2\kappa p_\oplus(1 - p_\oplus)}}\right),$$

consistent with Proposition 1.

Under the expected-occupancy and independent-overtaking approximations, the overtaking count is Poisson binomial with

$$\mu_{\text{shard}} = \frac{t}{M}\sum_{d=0}^{\kappa} N(d)P_{\text{overtake}}(d),$$

$$\sigma^2_{\text{shard}} = \frac{t}{M}\sum_{d=0}^{\kappa} N(d)P_{\text{overtake}}(d)\big(1 - P_{\text{overtake}}(d)\big).$$

When sufficiently many background items contribute and no individual term dominates the variance, $Rank_s \approx \mathcal{N}(1 + \mu_{\text{shard}}, \sigma^2_{\text{shard}})$.

Proof. Since every item is routed uniformly to t of the M shards, it appears in any fixed shard with probability t/M. Thus, a distance layer containing $N(d)$ background items contributes an expected $(t/M)N(d)$ items to that shard.

For a background item at clean distance d, the storage-side and query-side perturbations form an effective binary symmetric channel with crossover probability $p_\oplus$. Among the d initially different coordinates, the remaining mismatch count follows $X_d \sim \text{Binomial}(d, 1 - p_\oplus)$. Among the other $\kappa - d$ coordinates, newly created mismatches follow $Y_d \sim \text{Binomial}(\kappa - d, p_\oplus)$. Therefore, $\widetilde{D}_d = X_d + Y_d$, with mean $\kappa p_\oplus + d(1-2p_\oplus)$ and variance $\kappa p_\oplus(1-p_\oplus)$.

Under the standard independence approximation, the difference between the perturbed distances of the background item and the target has mean $(d - d_s^*)(1 - 2p_\oplus)$ and variance $2\kappa p_\oplus(1 - p_\oplus)$. Applying the Gaussian approximation gives $P_{\text{overtake}}(d)$. Hence, $I_{i,d} \sim \text{Bernoulli}(P_{\text{overtake}}(d))$, and the target rank is $Rank_s = 1 + \sum_d \sum_i I_{i,d}$.

Because the Bernoulli parameters depend on d, the overtaking count is Poisson binomial. Linearity of expectation gives $\mathbb{E}[Rank_s] = 1 + (t/M)\sum_d N(d)P_{\text{overtake}}(d)$. Under the independence approximation, its variance is $(t/M)\sum_d N(d)P_{\text{overtake}}(d)(1 - P_{\text{overtake}}(d))$, giving μ_{shard} and σ^2_{shard}. The central limit approximation for Poisson binomial variables then yields $Rank_s \approx \mathcal{N}(1 + \mu_{\text{shard}}, \sigma^2_{\text{shard}})$. □

The factor t/M above models expected shard occupancy for correctness. It is not treated as a subsampling probability in the complete-server-view privacy analysis.

Candidate Recovery. For a per-shard candidate budget P, the target-inclusion probability is approximately $P_{\text{hit},s}(P) = \Phi((P + \frac{1}{2} - (1 + \mu_{\text{shard}}))/\sigma_{\text{shard}})$, where $1/2$ is the continuity correction. If the experiment specifies a total candidate pool B distributed evenly across M shards, the per-shard budget is $\lfloor B/M \rfloor$; we write $P_{\text{hit},s}(B) = P_{\text{hit},s}(\lfloor B/M \rfloor)$.

For an item stored in the shard set $\mathcal{S}_i$, conditional independence across shards gives $P_{\text{hit},i}(B) \approx 1 - \prod_{s\in\mathcal{S}_i}(1 - P_{\text{hit},s}(B))$. Thus, the item is missed only when it is not recovered from any shard containing it.

The implementation returns candidates in multiples of M before duplicate removal. Therefore, the realized aggregate pool corresponding to configured budget B is $B_{\text{eff}}(B) = M\lfloor B/M \rfloor + 1/2$, including the continuity correction. The validation plots use this aggregate threshold to fit an effective normal-CDF model to the observed end-to-end metric. The parameters μ_R and σ_R reported in the plots are fitted aggregate-rank parameters; they need not equal the analytical shard-level moments above.

Recall Model. Absolute rank alone does not fully determine empirical Recall. The target must also be reached by HNSW traversal, returned in a shard-level candidate set, survive aggregation and duplicate removal, and enter exact reranking. Therefore, empirical Recall may saturate below one.

We introduce a coverage coefficient $\gamma_{\text{cov}} \in [0, 1]$ to capture these implementation-level effects. The fitted Recall curve is

$$R(B) = \gamma_{\text{cov}}\Phi\left(\frac{B_{\text{eff}}(B) - \mu_R}{\sigma_R}\right).$$

Here, γ_{cov} captures implementation-level candidate coverage and the empirical saturation level. It is not a parameter of the analytical shard-level rank distribution. The Recall panels of Figure 8 compare this model with the measurements.

MRR Model. For MS MARCO and TripClick, MRR@10 depends not only on whether a relevant document is recovered, but also on its final position after exact reranking. For a query q, let r_q be the rank of its first relevant document. Its reciprocal-rank contribution is $1/r_q$ when $r_q \le 10$ and zero otherwise. The dataset-level metric is $\text{MRR@10} = |Q|^{-1}\sum_{q\in Q}\text{RR@10}(q)$.

Let $H_q(B)$ be the probability that at least one relevant document is recovered, and let $c_q(B)$ denote the conditional expectation of $(1/r_q)\mathbf{1}\{r_q \le 10\}$ given this recovery event. Then

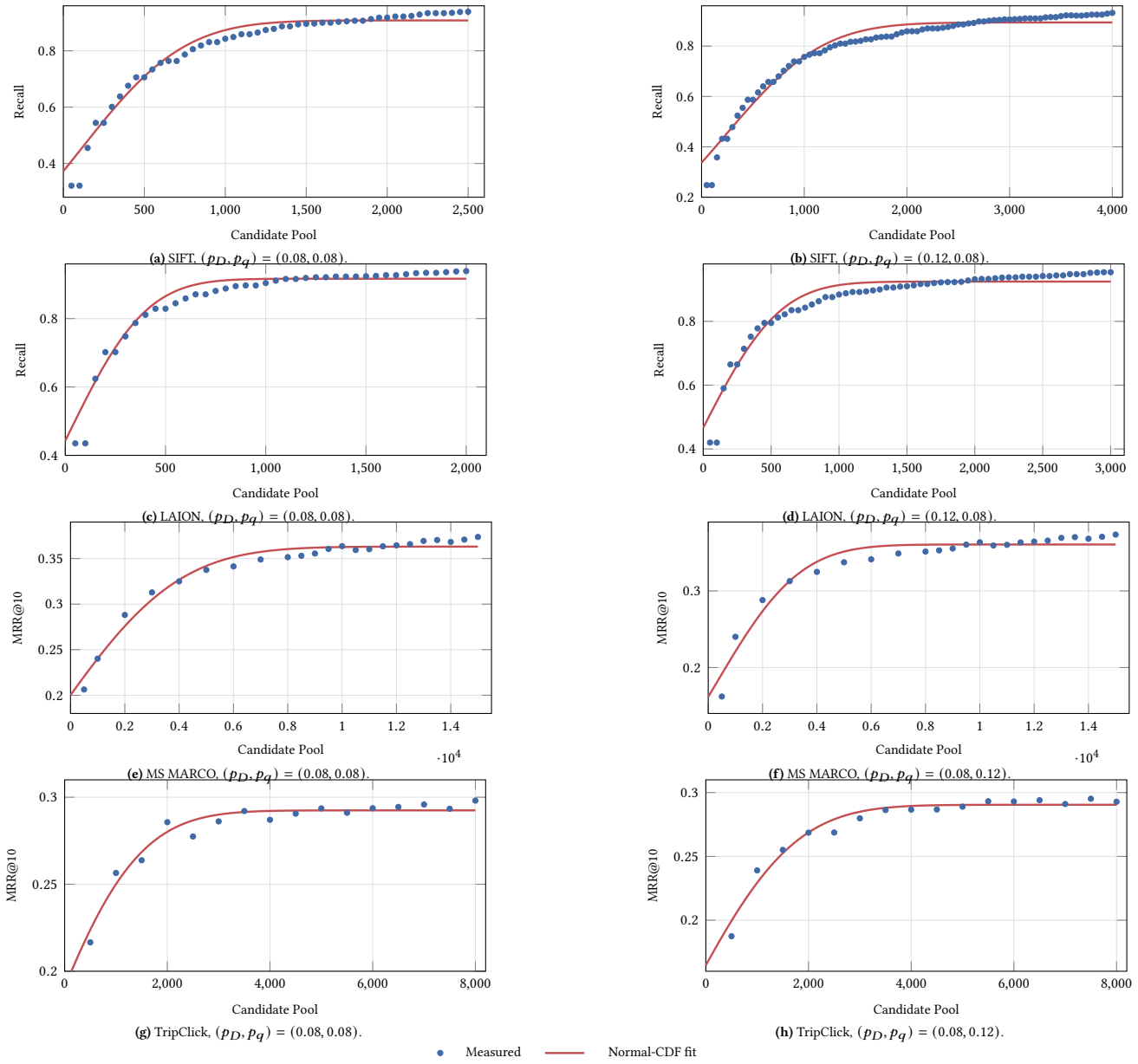
$$\mathbb{E}[\text{RR@10}(q)] = H_q(B)c_q(B).$$

Approximating the conditional ranking contribution by an empirical coefficient $\gamma_{\text{mrr}} \in [0, 1]$ gives the fitted curve

$$\text{MRR@10}(B) = \gamma_{\text{mrr}}\Phi\left(\frac{B_{\text{eff}}(B) - \mu_R}{\sigma_R}\right).$$

The coefficient γ_{mrr} captures empirical effects from candidate coverage, aggregation, duplicate removal, exact reranking, and the Top-10 evaluation window. Like γ_{cov}, it is not a parameter of the rank distribution or a formal correctness guarantee. The MRR panels of Figure 8 compare this model with the measurements. For every dataset and perturbation configuration, γ, μ_R, and σ_R are jointly estimated from all measured points by nonlinear least squares.

The derivation relies on Gaussian distance approximation, independent overtaking events, and conditional independence across selected shards. It therefore models the observed retrieval trends rather than exactly characterizing HNSW execution.



(a) SIFT, $(p_D, p_q) = (0.08, 0.08)$. **(b)** SIFT, $(p_D, p_q) = (0.12, 0.08)$.

(c) LAION, $(p_D, p_q) = (0.08, 0.08)$. **(d)** LAION, $(p_D, p_q) = (0.12, 0.08)$.

(e) MS MARCO, $(p_D, p_q) = (0.08, 0.08)$. **(f)** MS MARCO, $(p_D, p_q) = (0.08, 0.12)$.

(g) TripClick, $(p_D, p_q) = (0.08, 0.08)$. **(h)** TripClick, $(p_D, p_q) = (0.08, 0.12)$.

Fig. 8: Measured retrieval quality and fitted normal-CDF curves. Markers show measurements and lines show nonlinear least-squares fits over all measured points using $B_{\text{eff}}(B) = 64\lfloor B/64 \rfloor + 1/2$. Smooth continuous envelopes are drawn for readability.

Alg. 1 MDS-Based Cross-Shard Linkage

Server-visible shards $\mathsf{V} = \{\mathcal{G}^{(s)} = (V_s, E_s)\}_{s=1}^{M}$, where V_s and E_s are the entries and HNSW edges of shard s; perturbed code $\widetilde{\boldsymbol{v}}_{u,s}$ of each $u \in V_s$; fixed seed pairs $\mathcal{Z}_{s_0,s} \subseteq V_{s_0} \times V_s$; reference shard s_0 and target entry $a \in V_{s_0}$; MDS dimension r and threshold τ_{mds} Predicted corresponding-entry set $\widehat{C}_{\text{mds}}(a)$ and distinguishing score T_{mds}

for $s = 1, \ldots, M$ **do**
  Let D_s be the shard-local distance matrix, where $D_s[u, v] \leftarrow d_H(\widetilde{\boldsymbol{v}}_{u,s}, \widetilde{\boldsymbol{v}}_{v,s})$ and d_H denotes Hamming distance
  Let $X_s \leftarrow \textsc{ClassicalMDS}(D_s, r)$ be the resulting r-dimensional coordinates
end for
Initialize $\widehat{C}_{\text{mds}}(a) \leftarrow \{(s_0, a)\}$
Let $\mathcal{L}$ store the accepted matching scores associated with a
for $s \in [M] \setminus \{s_0\}$ **do**
  Use $\mathcal{Z}_{s_0,s}$ to obtain an orthogonal rotation R_s and translation $\boldsymbol{b}_s$
  Let $\widehat{X}_s \leftarrow X_s R_s + \mathbf{1}\boldsymbol{b}_s^T$ be the coordinates aligned with shard s_0
  for all $(u, v) \in V_{s_0} \times V_s$ **do**
    Define the MDS matching score as $S^{\text{mds}}_{s_0,s}(u, v) \leftarrow -\|X_{s_0}[u] - \widehat{X}_s[v]\|_2$
  end for
  Let $\pi_{s_0,s}$ be the one-to-one maximum-score assignment that preserves $\mathcal{Z}_{s_0,s}$
  Let $v^* \leftarrow \pi_{s_0,s}(a)$ be the entry assigned to a, or $\perp$ if no entry is accepted
  if $v^* \neq \perp$ and $S^{\text{mds}}_{s_0,s}(a, v^*) \geq \tau_{\text{mds}}$ **then**
    Add (s, v^*) to $\widehat{C}_{\text{mds}}(a)$
    Add $S^{\text{mds}}_{s_0,s}(a, v^*)$ to $\mathcal{L}$
  end if
end for
Let T_{mds} be the fixed scalar aggregation of the scores in $\mathcal{L}$
return $\widehat{C}_{\text{mds}}(a)$ and T_{mds}

Alg. 2 Topology-Based Cross-Shard Linkage

Server-visible shards $\mathsf{V} = \{\mathcal{G}^{(s)} = (V_s, E_s)\}_{s=1}^{M}$; fixed seed pairs $\mathcal{Z}_{s_0,s}$; reference shard s_0 and target entry $a \in V_{s_0}$; maximum hop count $h_{\max}$ and matching threshold τ_{top} Predicted corresponding-entry set $\widehat{C}_{\text{top}}(a)$ and distinguishing score T_{top}

for $s = 1, \ldots, M$ **do**
  for all $u \in V_s$ **do**
    Let $\boldsymbol{h}_s(u)$ be the topology descriptor containing the HNSW level, node degree, neighbor-degree statistics, and structural statistics within $h_{\max}$ hops
  end for
  Normalize all descriptors $\boldsymbol{h}_s(u)$ using fixed calibration statistics
end for
Initialize $\widehat{C}_{\text{top}}(a) \leftarrow \{(s_0, a)\}$
Let $\mathcal{L}$ store the accepted matching scores associated with a
for $s \in [M] \setminus \{s_0\}$ **do**
  for all $(u, v) \in V_{s_0} \times V_s$ **do**
    Define the initial topology score as $S^{\text{top}}_{s_0,s}(u, v) \leftarrow -\|\boldsymbol{h}_{s_0}(u) - \boldsymbol{h}_s(v)\|_2$
    Increase the score according to the fixed number of seed-supported neighboring correspondences
  end for
  Initialize the correspondence map $\pi_{s_0,s}$ with the fixed seeds $\mathcal{Z}_{s_0,s}$
  repeat
    Find mutually best unmatched pairs according to $S^{\text{top}}_{s_0,s}$
    Add pairs whose scores exceed τ_{top} to $\pi_{s_0,s}$ as pseudo-seeds
  until no new pair is added
  Let $v^* \leftarrow \pi_{s_0,s}(a)$ be the entry assigned to a, or $\perp$ if no entry is accepted
  if $v^* \neq \perp$ and $S^{\text{top}}_{s_0,s}(a, v^*) \geq \tau_{\text{top}}$ **then**
    Add (s, v^*) to $\widehat{C}_{\text{top}}(a)$
    Add $S^{\text{top}}_{s_0,s}(a, v^*)$ to $\mathcal{L}$
  end if
end for
Let T_{top} be the fixed scalar aggregation of the scores in $\mathcal{L}$
return $\widehat{C}_{\text{top}}(a)$ and T_{top}

Alg. 3 Graph-Signal-Based Cross-Shard Linkage

Server-visible shards $\mathsf{V} = \{\mathcal{G}^{(s)} = (V_s, E_s)\}_{s=1}^{M}$; perturbed code $\widetilde{v}_{u,s}$ of each $u \in V_s$; fixed seed pairs $\mathcal{Z}_{s_0,s}$; reference shard s_0 and target entry $a \in V_{s_0}$; filter depth H, filter coefficient α, and matching threshold τ_{gsgm} Predicted corresponding-entry set $\widehat{C}_{\text{gsgm}}(a)$ and distinguishing score T_{gsgm}

for $s = 1, \ldots, M$ **do**
  Let A_s be the adjacency matrix of $\mathcal{G}^{(s)}$
  Let $\widehat{A}_s$ be the degree-normalized adjacency matrix derived from A_s
  Let $F_s^{(0)}$ be the initial node-signal matrix obtained from fixed summaries of the perturbed binary embeddings
  for $h = 1, \ldots, H$ **do**
    Compute the signal matrix after the h-th filtering step as $F_s^{(h)} \leftarrow \alpha F_s^{(0)} + (1-\alpha)\widehat{A}_s F_s^{(h-1)}$
  end for
  for all $u \in V_s$ **do**
    Let $\boldsymbol{g}_s(u)$ be the graph-signal descriptor obtained by concatenating $F_s^{(0)}[u], \ldots, F_s^{(H)}[u]$
  end for
end for
Initialize $\widehat{C}_{\text{gsgm}}(a) \leftarrow \{(s_0, a)\}$
Let $\mathcal{L}$ store the accepted matching scores associated with a
for $s \in [M] \setminus \{s_0\}$ **do**
  Use $\mathcal{Z}_{s_0,s}$ to align $\boldsymbol{g}_s$ with the graph-signal descriptor space of shard s_0
  Denote the aligned descriptor by $\widehat{\boldsymbol{g}}_s$
  for all $(u, v) \in V_{s_0} \times V_s$ **do**
    Define the graph-signal score as $S_{s_0,s}^{\text{gsgm}}(u, v) \leftarrow -\|\boldsymbol{g}_{s_0}(u) - \widehat{\boldsymbol{g}}_s(v)\|_2$
  end for
  Let $\pi_{s_0,s}$ be the one-to-one maximum-score assignment that preserves $\mathcal{Z}_{s_0,s}$
  Let $v^* \leftarrow \pi_{s_0,s}(a)$ be the entry assigned to a, or $\perp$ if no entry is accepted
  if $v^* \neq \perp$ and $S_{s_0,s}^{\text{gsgm}}(a, v^*) \geq \tau_{\text{gsgm}}$ **then**
    Add (s, v^*) to $\widehat{C}_{\text{gsgm}}(a)$
    Add $S_{s_0,s}^{\text{gsgm}}(a, v^*)$ to $\mathcal{L}$
  end if
end for
Let T_{gsgm} be the fixed scalar aggregation of the scores in $\mathcal{L}$
return $\widehat{C}_{\text{gsgm}}(a)$ and T_{gsgm}

Alg. 4 Cycle-Consistent Joint Cross-Shard Linkage

Pairwise scores $S_{s,r}^{\mathcal{A}}(u, v)$ obtained by applying the scoring stages of MDS, topology, and GSGM to every shard pair, where $\mathcal{A} \in \{\text{mds}, \text{top}, \text{gsgm}\}$; calibration mean $\mu_{\mathcal{A}}$, standard deviation $\sigma_{\mathcal{A}}$, and fixed weight $\omega_{\mathcal{A}}$ for each method; fixed seed pairs $\mathcal{Z}_{s,r}$; reference shard s_0 and target entry a; thresholds τ_{joint} and τ_{cyc}; maximum iteration count $I_{\max}$ Jointly predicted corresponding-entry set $\widehat{C}_{\text{joint}}(a)$ and distinguishing score T_{joint}

for all shard pairs (s, r) **do**
  for all candidate pairs $(u, v) \in V_s \times V_r$ **do**
    for all $\mathcal{A} \in \{\text{mds}, \text{top}, \text{gsgm}\}$ **do**
      Let $\overline{S}_{s,r}^{\mathcal{A}}(u, v) \leftarrow \frac{S_{s,r}^{\mathcal{A}}(u,v) - \mu_{\mathcal{A}}}{\sigma_{\mathcal{A}}}$ be the standardized score of method $\mathcal{A}$
    end for
    Define the combined score as $S_{s,r}^{\text{joint}}(u, v) \leftarrow \sum_{\mathcal{A}} \omega_{\mathcal{A}} \overline{S}_{s,r}^{\mathcal{A}}(u, v)$
  end for
  Let $\pi_{s,r}$ be the one-to-one maximum-score assignment that preserves $\mathcal{Z}_{s,r}$
end for
for $i = 1, \ldots, I_{\max}$ **do**
  Let changed indicate whether a new pseudo-seed is added in this iteration
  changed $\leftarrow$ false
  for all predicted pairs $v = \pi_{s,r}(u)$ **do**
    Let $\rho_{s,r}(u, v)$ be the fraction of available third shards g for which the indirect mapping satisfies $\pi_{g,r}(\pi_{s,g}(u)) = v$
    if $S_{s,r}^{\text{joint}}(u, v) \geq \tau_{\text{joint}}$ and $\rho_{s,r}(u, v) \geq \tau_{\text{cyc}}$ **then**
      if (u, v) is not already a seed **then**
        Add (u, v) as a pseudo-seed
        changed $\leftarrow$ true
      end if
    end if
  end for
  if not changed **then**
    break
  end if
  Recompute each affected assignment $\pi_{s,r}$ using the updated seeds
end for
Initialize $\widehat{C}_{\text{joint}}(a) \leftarrow \{(s_0, a)\}$
Let $\mathcal{L}$ store the accepted joint matching scores associated with a
for $s \in [M] \setminus \{s_0\}$ **do**
  Let $v^* \leftarrow \pi_{s_0,s}(a)$ be the entry jointly assigned to a
  if $v^* \neq \perp$, $S_{s_0,s}^{\text{joint}}(a, v^*) \geq \tau_{\text{joint}}$, and $\rho_{s_0,s}(a, v^*) \geq \tau_{\text{cyc}}$ **then**
    Add (s, v^*) to $\widehat{C}_{\text{joint}}(a)$
    Add $S_{s_0,s}^{\text{joint}}(a, v^*)$ to $\mathcal{L}$
  end if
end for
Let T_{joint} be the fixed scalar aggregation of the scores in $\mathcal{L}$
return $\widehat{C}_{\text{joint}}(a)$ and T_{joint}